Resonator-Factored Hierarchical Hypervector Embeddings for Compact Structured Memory and Faster Factor-Aligned Computation:

General Theory and Application to Columnar Causal Networks and Neural-Symbolic Associative Memory

Ben Goertzel

July 25, 2026

Abstract

We describe a family of resonator-decoded hierarchical hypervector codes for approximately factorized data and learned latent structures. We then show that, when a neural or algorithmic system exposes a bounded-load factor graph for a specified task family, vector-symbolic or hyperdimensional computing (HDC) can provide a compact, compositional, queryable cache of those factors.

The dimension required for reliable local queries depends on local crosstalk load, effective codebook size, codebook coherence, cleanup margin, query depth, and the number of supported queries, but not directly on the raw number of pixels, frames, tokens, or leaves. We make this precise using ordered roles, finite or factorized level codebooks, query-limited capacity bounds, a task-specific factorization defect, coherence-aware cleanup bounds, and a basin-conditional resonator margin theorem. The coherence-aware form of the bounds is important in practice: for recursively constructed subtree dictionaries containing near-duplicate entries, the dimension requirement degrades from $D \gtrsim k \log M$ to $D \gtrsim k^{3/2} \log M$ (bipolar sign codes, via an arcsine-law decorrelation) or $D \gtrsim k^2 \log M$ (linear and phasor codes), and we treat this explicitly as a design constraint rather than hiding it in constants. A toy-scale numerical study included in the paper confirms these scaling laws, the polynomial cold-start operational capacity of resonator decoding together with its rescue by cue-based basin entry, and the necessity of argument-position permutation for directed relations; a companion real-data example (quadtree-factorized handwritten digits with learned k-means codebooks) shows that observed subtree dictionaries are small but pervasively coherent, and that when the query-time encoder is noisy relative to the stored dictionaries, the cleanup step itself can become net harmful below the coherence-aware dimension budget, making encoder–dictionary consistency a precondition for cleanup. A further real-data study on spoken digits replicates these findings in the temporal domain, decodes two query families (word and speaker) from one code, and shows that a role-free commutative code encodes an utterance and its time-reversal identically, so that temporal direction survives exactly insofar as ordered roles are enforced. A final study applies the framework to streaming generation over a residual-quantized audio codec, finding that a single role-bound output projection with parallel cleanup matches independent per-codebook heads, that temporal persistence supplies resonator basin entry within a real-time frame budget, and that the value of fixed-width history compression is governed by a factorization defect that toy-scale data cannot settle.

Next, we suggest to turn the HDC layer from a passive compression stage into an active guide for representation learning. The HDC capacity law tells the upstream learner what kind of factorization to seek: reusable, role-stable, locally decodable, low-load, high-margin, well-separated factors with little task-relevant information left in the residual. We formulate HDC-admissibility

diagnostics and losses for effective local load, cleanup margin, confusable codebook size, dictionary coherence, product recomposition, role consistency, residual predictive information, and intervention locality. This idea applies beyond DNNs to any structured algorithm whose internal variables can be aligned with HDC roles and codebooks.

We also show how this same machinery can be combined with Hierarchical Modular Hypervectors (HMH) and PRIMUS-style world modeling to yield resonant modular compositional episodic memory. In this variant, an episode is stored not as a monolithic trace but as a typed, role-filler, evidence-anchored factor graph: actors, objects, places, times, goals, affective and social context, causal links, affordance trials, outcomes, skills, and perceptual pointers are bound into ontology-aligned modules, with ordered argument roles so that directed relations such as causation and temporal precedence remain directed under commutative binding. Resonator cleanup then supports partial structured recall, slot completion, counterfactual factor swaps, schema consolidation, and neural judgment over densified HMH traces in a way that respects the factorization of the remembered episode.

Finally, we instantiate the framework in Hierarchical Bayesian Causal Modular Learning (HBCML) and Columnar Bayesian Causal Coding (ColBaC), a two-level columnar continual-learning architecture. In ColBaC, sparse Bayesian support selection, internal column certificates, hard kernels, adaptable shells, typed microcolumns, and local counterfactual teachers give an especially direct substrate for HDC-guided factorization. ColBaC learns and protects causal columns; the HDC layer makes their reusable contents addressable, compositional, auditable, and capacity-aware.

AI Acknowledgement Lots of help from GPT-5.5-Pro, Claude Opus 4.7, Claude Fable 5 and Gemini Pro

Contents

1 Introduction

Images, video, language, program traces, relational records, and agent memories are high-dimensional objects. They are not, in general, losslessly compressible into one fixed-width vector. Yet most useful computations over them never require full reconstruction. They require local access to object states, roles, events, spans, mechanisms, retrieved passages, syntax subtrees, options, or other reusable latent factors. This paper studies a representation layer for precisely that regime: a compact structured memory that one can store cheaply, query locally, and—when the memory's factorization is aligned with the host model's own factor structure—compute over directly, rather than re-deriving it from raw data.

The intended pipeline is

$$x \xrightarrow{E_\theta} z(x) \xrightarrow{F_\psi} G_x \xrightarrow{\text{Enc}} H_x \in \mathbb{H}_D. \tag{1}$$

Here x is raw data; E_θ is a perceptual, linguistic, causal, or algorithmic encoder; $z(x)$ is its full internal state; F_ψ extracts a roughly factorized latent graph G_x; and H_x is a fixed-width vector-symbolic code, or hyperdimensional embedding, of that graph. The HDC layer is not asked to solve perception or language modeling from scratch. It is asked to store and query a structured latent graph that another system has already learned or constructed, in a form whose algebra matches the structure of the host computation.

The central distinction is between all-information reconstruction and query-faithful representation. A code that answered every possible leaf query for an N-leaf object with uniformly high probability would permit reconstruction of the whole object, and must pay the usual entropy cost. A code that supports a fixed or slowly growing family of local, role-bound, or homomorphic queries can be far smaller. The dimension that controls reliability is set by the local interaction load, the effective number of confusable states at each cleanup step, and the geometric separation of those states, not by raw object size alone.

This is also where the computational payoff comes from. When the data is factorized in a way that matches the factor structure the host model already uses, a query can be answered, and a reusable factor retrieved or recombined, by operating on the compact code instead of scanning the

raw object or re-running the encoder. Two cost regimes matter. First, resonator factorization recovers an unknown product of factors by iterated slot-wise cleanup at cost proportional to $\sum_f |C_f|$, rather than by brute-force search over the product space $\prod_f |C_f|$; this is the exponential-to-linear win, and it is conditional on a measurable margin *and* on entering the resonator's basin of attraction, whose search overheads must be charged. Second, a local role-bound query touches only the few factors on its path, so its cost scales with query depth and local load rather than with object size, and it can replace a raw or full-latent scan, dense attention over stored contexts, or energy-intensive neural recomputation. We are deliberate about the limits of this claim: a single local HDC lookup does not asymptotically beat a well-engineered indexed tree, and when the encoder E_θ must run at query time its compute is charged separately from the cached code. The speedups are real, but they are regime-specific and live in the memory, query, and reuse layer—and, in columnar architectures, in the operation of the columns themselves.

The two error sources separate cleanly:

$$L_{\text{HDC}} - L_{\text{full}} = \underbrace{L_{\text{fact}} - L_{\text{full}}}_{\text{factorization defect}} + \underbrace{L_{\text{HDC}} - L_{\text{fact}}}_{\text{HDC coding error}} . \tag{2}$$

The HDC theory controls the second term through dimension, margin, coherence, and local load. It does not control the first. If the host system relies on holistic, distributed synergies that no bounded-load factor graph can approximate for the chosen task family, the defect term stays large even as $D \to \infty$. This is not a loophole; it is the main empirical diagnostic for whether a structured hypervector memory is the right tool at all.

A second theme is that the HDC layer need not be a passive cache placed after the host model. The same capacity law that says when a code is reliable also says which factorizations are cheap to store and fast to query: reusable, role-stable, locally decodable, low-load, high-margin, well-separated factors that leave little task-relevant information in the residual. Read in that direction, the capacity law becomes a learning objective. The host learner is pressed not merely to compress, but to factorize in a way that supports cheap binding, bundling, cleanup, resonator factorization, and role-conditioned query. We formulate the corresponding admissibility diagnostics and losses, and note that they apply to any structured algorithm—not only deep nets—whose internal variables can be aligned with HDC roles and codebooks.

This same idea also gives a useful way to reinterpret episodic memory. Episodic traces in agents are not merely bags of percepts, and not merely timestamps on raw observations. They typically have a semi-factorized organization: who did what to whom, where, when, for what reason, under what affordance, social, normative, or causal constraints, with what evidence, and with what outcome. Hierarchical Modular Hypervectors (HMH) already provide a typed modular role-filler representation for such structures, while PRIMUS-style world modeling uses HMH as a structured associative regime between symbolic atoms and dense neural tensors. Adding resonator cleanup to this substrate yields what we will call resonant modular compositional episodic memory: an episode can be stored as an HMH-like factor graph, retrieved by partial structured cues, completed by slot-wise resonator cleanup, densified for neural judgment, and consolidated into semantic schemas, affordance templates, social scripts, or reusable skills. This is not a separate application bolted onto the paper's theory; it is a direct case of the general query-limited story, with episodes playing the role of structured latent graphs and episodic questions playing the role of supported local queries.

This perspective is especially natural in columnar causal architectures, where we develop it concretely. Hierarchical Bayesian Causal Modular Learning (HBCML) decomposes continual learning into a sparse top-level controller that selects structurally restricted components and an internal

probabilistic process that separates reusable causal structure from task-local residue. Columnar Bayesian Causal Coding (ColBaC) is a concrete columnar instance with hard kernels, adaptable shells, typed microcolumns, internal certificates, and local one-swap teachers. These architectures already expose the objects a structured memory needs—supports, columns, shells, motifs, certificates, options—so HDC can supply the algebraic language in which their reusable contents become addressable, compositional, auditable, and capacity-aware. Here the cache-level speedups and the cost of operating the network largely coincide, which is why ColBaC is our principal application rather than an afterthought.

The paper makes four design commitments. First, descent through a hierarchy uses explicit level dictionaries or factored dictionaries of possible child states; a dictionary of all formal subtrees is intractably large, so what matters is the effective metric entropy of the task-relevant learned states, together with their geometric separation. Second, binding is either a fixed vector-symbolic algebra, such as MAP or phasor HRR, or a learned operation carrying an explicit quasi-isometry penalty. Third, resonator decoding is claimed only under a measurable margin condition together with an explicit basin-entry condition, with all search overheads charged. Fourth, speedup is always stated against a named baseline: an HDC local query does not asymptotically beat a conventional indexed tree lookup, but it can beat raw scans, full latent scans, dense attention over stored contexts, brute-force factor search, or neural recomputation in the appropriate regime.

Contributions. The paper contributes: (i) a query-limited theory of hierarchical resonator HDC codes with explicit level codebooks, ordered roles, coherence-aware cleanup bounds, basin-conditional resonator margins, a task-specific factorization defect, and honest storage and compute accounting; (ii) a compute model that ties compression to query speed and identifies the regimes in which factor-aligned hypervector computation is genuinely cheaper than scanning, dense attention, brute-force factor search, or recomputation; (iii) a set of scalability criteria that turn common objections into measurable quantities, including an explicit treatment of near-duplicate subtree dictionaries and correlated crosstalk; (iv) an HDC-guided learning objective that pushes a host DNN or structured algorithm toward capacity-favorable factorizations; (v) a resonant HMH construction for episodic memory and PRIMUS-style world modeling, with ordered argument roles for directed relations, showing how factorized episodes can be retrieved, completed, densified, judged, and consolidated while preserving their role-filler structure; and (vi) a detailed instantiation in ColBaC/HBCML showing how HDC requirements can shape column selection, internal certificates, shell hygiene, support memory, and causal modularity.

2 Vector-symbolic algebra

The basic idea of a vector-symbolic architecture [1, 2, 3, 4] is that a single long vector can carry structured, symbol-like content, with a small algebra of operations that mirror the operations one would perform on symbols. Atoms play the role of distinct symbols; because they are random and high-dimensional, distinct atoms are nearly orthogonal, and this near-orthogonality is what lets many of them coexist in one vector without colliding.

Let $\mathbb{H}_D$ be a D-dimensional hypervector space. Examples include bipolar MAP vectors $\{-1, +1\}^D$ with elementwise multiplication, complex phasor HRR vectors with elementwise phase multiplication, or normalized real vectors with an approximately unitary binding operation. We use four operations.

- **Binding** $u \otimes v$ composes two vectors; intuitively it forms an association or key–value tag. For atom a, binding by a is approximately an isometry and has an inverse $a^\dagger$ satisfying $a^\dagger \otimes a = 1$.

The bound vector is dissimilar to both of its inputs, so binding hides content; unbinding by the inverse, $a^\dagger \otimes (a \otimes v) = v$, recovers the partner.

- **Bundling** $u \oplus v$ superposes vectors; intuitively it forms an unordered set or memory trace. In most implementations this is ordinary addition followed by an optional normalization or quantization. Unlike binding, a bundle stays similar to each of its summands, so set membership can be tested by similarity, at the cost of crosstalk that grows with the number of superposed terms.
- **Permutation** ρ or depth-specific roles encode order. We do not rely on commutativity to distinguish paths: applying a permutation (or using a position-specific role atom) makes the contribution of one slot or path nearly independent of the others, so that, for example, (r_1, r_2) and (r_2, r_1) are not conflated.
- **Cleanup** maps a noisy vector to a nearby codebook element; it is the error-correction step that pulls an approximate estimate back onto a legal symbol.

A finite codebook is $C = \{c_1, \ldots, c_M\} \subset \mathbb{H}_D$, also viewed as a $D \times M$ matrix. Soft and hard cleanup are

$$\mathrm{cleanup}_C(z) = C\,\mathrm{softmax}(\beta C^\top z), \qquad \widehat{\mathrm{cleanup}}_C(z) = \arg\max_{c \in C} \langle c, z \rangle. \tag{3}$$

Hard cleanup is nearest-neighbor decoding against the codebook; soft cleanup is its smoothed, differentiable relaxation, a softmax-weighted blend of codebook entries that sharpens toward the hard rule as $\beta \to \infty$. Soft cleanup is useful for differentiable or iterative updates; final readout uses the hard rule.

For soft intermediate states we also need a projection back to the algebraic state space before inversion:

$$\mathrm{Proj}_{\mathbb{H}}(z) = \begin{cases} \mathrm{sign}(z), & \text{bipolar MAP,} \\ z/|z|, & \text{phasor HRR, componentwise when } z_i \neq 0, \\ z/\|z\|, & \text{unit-sphere real model.} \end{cases} \tag{4}$$

A soft, softmax-weighted blend of atoms is not itself a legal element of the binding algebra, so unbinding it directly is not well defined; the projection snaps the estimate back onto the manifold (signs, unit phases, or unit norm) where binding inverses behave properly. Algorithmic inversion is therefore applied to $\mathrm{Proj}_{\mathbb{H}}(z)$, not to an arbitrary convex combination of atoms.

Assumption 1 (Random atoms and concentration)**.** Atoms are independent normalized random hypervectors with $\|a\|^2 = D$, $\mathbb{E}\langle a, a' \rangle = 0$ for independent $a \neq a'$, and sub-Gaussian normalized overlap: for some constant $c_0 > 0$,

$$\mathbb{P}\big[\, |\langle a, a' \rangle| \geq tD \,\big] \leq 2\exp(-c_0 D t^2), \qquad t \in (0, 1]. \tag{5}$$

Binding by an independent atom preserves these concentration bounds.

Assumption 2 (Isometric binding)**.** For every atom a, $\langle a \otimes u, a \otimes v \rangle = \langle u, v \rangle$, and binding distributes over bundling before normalization. This holds exactly in common idealized MAP and Fourier-domain HRR models and approximately in many learned VSA layers.

Assumption 3 (Role decorrelation of reused content)**.** The same content atom may appear many times in one code or in one level dictionary. When two occurrences of a shared atom c appear under *distinct* role products, e.g. $r_i \otimes c$ and $r_j \otimes c$ with $r_i \neq r_j$ independent role atoms, the two occurrences are treated as decorrelated: by Assumptions 1–2, $\langle r_i \otimes c, r_j \otimes c \rangle = \langle r_i, r_j \rangle$ (in idealized MAP), which concentrates near zero. Reuse of content under *identical* role products is not decorrelated; it produces genuinely coherent (structurally overlapping) codebook entries and crosstalk, and is handled explicitly through the coherence parameter of Section 5 rather than through independence assumptions.

Assumption 3 makes explicit a point that pure random-atom analyses gloss over: recursively constructed codes reuse atoms, so the independence in Assumption 1 cannot be invoked blindly. Role tagging restores near-orthogonality across role positions; what remains, and what the theory must track, is same-role structural overlap between distinct dictionary entries.

Ordering convention. A commutative binding algebra cannot by itself distinguish the path (r_1, r_2) from (r_2, r_1), nor the directed relation $\lambda(a, b)$ from $\lambda(b, a)$. We therefore use either depth-specific role atoms $r_{t,i}$ or role atoms composed with a permutation, e.g. $r_i \otimes \rho(H_{\text{child}})$; for directed relations we apply the permutation (or a dedicated argument-position role) to exactly one argument. The theory below assumes that two distinct ordered paths, and the two orientations of any directed relation, have independent or near-independent role products.

3 Learned factor graphs

3.1 From raw data to a quantized latent graph

Let $x \in \mathcal{X}$ be a raw datum: an image, video clip, audio segment, document, program, molecule, or relational record. A neural encoder E_θ produces a latent state $z(x)$. A factorizing module F_ψ maps $z(x)$ to a finite graph or tree

$$G_x = (V_x, E_x), \tag{6}$$

with maximum depth L and maximum local branching b_t at level t. The graph may be a true tree, a tree decomposition of a graph, a scene graph, an object track hierarchy, a parse hierarchy, a multiscale patch hierarchy, or a causal state graph unfolded over time. The theory is simplest for trees. DAGs can be handled by duplicating shared nodes, using junction trees, or adding explicit cross-edge roles and treating those edges as additional local load.

Each node $\nu \in V_x$ has a latent state z_ν. At level $t(\nu)$, a quantizer or dictionary learner assigns z_ν to a finite prototype codebook

$$K_t = \{k_{t,1}, \ldots, k_{t,M_t}\} \subset \mathbb{H}_D. \tag{7}$$

The prototype may be an atom, a normalized subtree prototype, or a product of factor atoms

$$u_\nu = c_{\nu,1} \otimes \cdots \otimes c_{\nu,F_t}, \qquad c_{\nu,f} \in C_{t,f}. \tag{8}$$

In DNN applications the codebooks K_t and $C_{t,f}$ can be produced by VQ layers, product quantization, slot prototypes, sparse autoencoder dictionaries, object-state prototypes, or causal-state prototypes. The codebook sizes M_t are part of the theorem. If M_t grows exponentially with object size, then the HDC dimension must grow as well; the dimension is independent of raw size only when the learned latent uses reusable local prototypes.

Definition 1 (Neural factorization defect)**.** Fix a task or query family Q. Each query $q \in Q$ comes with a common answer space $\mathcal{Y}_q$, an answer map on full latents $A_q^{\text{full}} : z \mapsto A_q^{\text{full}}(z) \in \mathcal{Y}_q$, an answer map on factor graphs $A_q^{\text{fact}} : G \mapsto A_q^{\text{fact}}(G) \in \mathcal{Y}_q$, and a loss $d_q : \mathcal{Y}_q \times \mathcal{Y}_q \to [0, B_q]$. The factorization defect of the learned graph family $\mathcal{G} = \{G_x\}$ relative to the full neural state $z(x) = E_\theta(x)$ is

$$\delta_Q = \sup_{q \in Q} \mathbb{E}_x \Big[d_q\big(A_q^{\text{full}}(z(x)),\, A_q^{\text{fact}}(G_x)\big) \Big]. \tag{9}$$

The defect includes quantization error, missing nonlocal interactions, and any information in $z(x)$ that the factor graph does not preserve for the query family. It is not assumed to be an energy ratio and need not be the same for all tasks. Fixing the shared answer space $\mathcal{Y}_q$ is what makes the comparison between the two answer maps well defined; "the same query applied to two different objects" is otherwise not a meaningful expression.

This definition is deliberately task-specific. A video factor graph may have low defect for object tracking and high defect for photorealistic frame reconstruction. A document factor graph may have low defect for entity retrieval and high defect for open-ended generation. A language-model sparse feature graph may have low defect for a probing task and high defect for full next-token prediction.

3.2 Supported query model

Let Q_{sup} be the family of queries the code is designed to answer. A query may specify one or more root-to-node paths, a set of roles, a factor slot to recover, or a summary readout. Let

$$q_{\max} = |Q_{\text{sup}}|, \qquad \ell_{\max} = \max_{q \in Q_{\text{sup}}} \text{depth touched by } q. \tag{10}$$

The local capacity theorem below is for a fixed supported query family. The queries can be chosen after seeing the training distribution and codebooks, but not adversarially after seeing the random atom draw unless the margin conditions are verified for that draw.

Remark 1 (Why the query parameter is necessary). If a constant-dimensional code could answer every one of N leaf queries with high probability independent of N, then querying all leaves would reconstruct the object. Thus any theorem claiming dimension independent of N must be query-limited, distributional, approximate, or nonuniform. We make this explicit through $q_{\max}$ and through the all-path lower bound in Theorem 2.

4 Encoder and query algorithms

4.1 Hierarchical code

For a node ν at level t, let $u_\nu \in K_t$ be its quantized local state, possibly factored as in Eq. (8). Let $r_{t,i}$ be the ordered role for child position i at level t, τ_t a role for the local node state, and σ_t an optional summary role. The recursive code is

$$H_\nu = \text{Normalize}\left(\tau_t \otimes u_\nu \;\oplus\; \bigoplus_{i=1}^{\deg(\nu)} r_{t,i} \otimes \rho(H_{\text{child}_i(\nu)}) \;\oplus\; \sigma_t \otimes S_\nu \right). \tag{11}$$

The intuition is that a node's code is a single superposition of three kinds of tagged content: the node's own local state, tagged by the state role τ_t; each child's recursively encoded subtree, tagged

both by which child slot it fills ($r_{t,i}$) and by a permutation ρ that marks depth and prevents sibling paths from colliding; and an optional summary channel for whole-subtree readouts. Because the children enter through distinct role tags, a later query can recover any one of them by unbinding the corresponding role, while the other children act only as bounded crosstalk. Here S_ν is a learned or hand-built summary vector for subtree-level readouts. The normalization Normalize specifies whether bundles remain real sums, are projected to bipolar signs, are projected to phasors, or are otherwise quantized. The same choice must be used in the capacity and storage model.

The level codebook used during descent is not implicit. It is the finite set of valid normalized child encodings at that level,

$$K_t^{\text{sub}} = \{\text{possible } H_\nu \text{ for nodes at level } t\}, \tag{12}$$

or a learned approximation to that set. In practice this can be a VQ dictionary of subtree states, a product-quantized dictionary, a cache of prototypes learned from training latents, or a generated dictionary whose seed and architecture are counted in storage. The theoretical bounds use $M_t^{\text{sub}} = |K_t^{\text{sub}}|$. Because entries of K_t^{sub} are built recursively from shared child codes, they are *not* independent random atoms; two subtrees differing in a single child overlap structurally. Section 5 tracks this through an explicit coherence parameter.

Algorithm 1 Encode a learned factor tree

procedure ENCODE(ν, level t)
  $u_\nu \leftarrow \text{Quant}_t(z_\nu)$ ▷ prototype in K_t, or product over factor codebooks
  $H \leftarrow \tau_t \otimes u_\nu$
  for $i = 1$ to $\deg(\nu)$ **do**
    $C_i \leftarrow \text{ENCODE}(\text{child}_i(\nu), t+1)$
    $H \leftarrow H \oplus (r_{t,i} \otimes \rho(C_i))$
  end for
  $S_\nu \leftarrow \text{Summarize}(\nu)$ ▷ optional task-preserving digest
  $H \leftarrow H \oplus (\sigma_t \otimes S_\nu)$
  return Normalize(H)
end procedure

Algorithm 2 Descend along a known ordered path

procedure DESCEND(H, path $(i_1, \ldots, i_\ell)$)
  $h \leftarrow H$
  for $t = 0$ to $\ell - 1$ **do**
    $h \leftarrow \rho^{-1}(r_{t,i_{t+1}}^{\dagger} \otimes h)$ ▷ target child plus sibling crosstalk
    $h \leftarrow \widehat{\text{cleanup}}_{K_{t+1}^{\text{sub}}}(h)$ ▷ explicit level-codebook cleanup
  end for
  return h
end procedure

4.2 Path descent

Descent walks down a known path one level at a time. Each step first unbinds the role of the chosen child, which exposes that child's subtree code superposed with residual crosstalk from its siblings,

and then cleans the result against the level's subtree dictionary to discard that crosstalk and recover a legal subtree code. Without the cleanup on line 5, crosstalk accumulates from off-path subtrees and the effective load can scale with the whole object. With the cleanup and a valid K_t^{sub}, each step faces only local load—subject to the coherence caveat of Section 5.1: when the dictionary contains near-duplicate entries, cleanup may return a confusable neighbor, and the resulting error surfaces only when the query later descends into the branch where the neighbors differ.

4.3 Resonator factorization

Intuitively, the resonator [6, 7] factors a product by alternating estimation: it holds all but one factor fixed at their current guesses, unbinds them from H to isolate the evidence bearing on the remaining factor, cleans that evidence up against the corresponding codebook, and cycles through the factors until the hard estimates stop changing. The payoff is that each sweep searches the individual factor codebooks separately, at cost proportional to $\sum_f |C_f|$, rather than enumerating the full product space $\prod_f |C_f|$.

The projection step is important: a soft superposition is not generally an invertible atom in MAP or HRR. One can strengthen the practical decoder with multi-start initialization, top-K beams, damping, temperature annealing, and a recomposition score

$$\langle H, \hat{x}_1 \otimes \cdots \otimes \hat{x}_F \rangle \tag{13}$$

to reject spurious fixed points.

Algorithm 3 Margin-safe resonator factorization of $H \approx x_1 \otimes \cdots \otimes x_F$

1: **procedure** FACTORIZE(H, codebooks $C_1, \ldots, C_F$, iterations I)
2: **for** $f = 1$ to F **do**
3: $\hat{x}_f \leftarrow \mathrm{Normalize}\big(\sum_{c \in C_f} c\big)$
4: **end for**
5: **for** $s = 1$ to I **do**
6: **for** $f = 1$ to F **do**
7: $y_g \leftarrow \mathrm{Proj}_{\mathbb{H}}(\hat{x}_g)$ for all $g \neq f$
8: $z_f \leftarrow H \otimes \big(\bigotimes_{g \neq f} y_g^{\dagger}\big)$
9: $\hat{x}_f \leftarrow \mathrm{cleanup}_{C_f}(z_f)$
10: **end for**
11: **if** all hard cleanups are unchanged **then**
12: **break**
13: **end if**
14: **end for**
15: **return** $\big(\widehat{\mathrm{cleanup}}_{C_1}(\hat{x}_1), \ldots, \widehat{\mathrm{cleanup}}_{C_F}(\hat{x}_F)\big)$
16: **end procedure**

4.4 Structured queries

A query can use three access modes.

1. A local path query runs DESCEND, then optionally runs FACTORIZE on the returned node state.

2. A summary query reads a summary channel $\sigma_t \otimes S_\nu$.

3. A homomorphic readout applies a linear or role-equivariant map that has been defined on the encoded representation itself.

The third case needs care. A generic cosine similarity against a content atom at the root is not automatically a count of that content in the underlying tree, because leaf contents are role-bound by their paths. Valid global readouts are linear maps trained directly on root codes, summary-channel readouts explicitly constructed to preserve the statistic, or role-equivariant maps satisfying a known relation such as $\phi(r \otimes x) = A_r \phi(x)$.

Algorithm 4 Structured query

procedure StructuredQuery(H, query specification q)
  if q is a valid root summary or learned linear readout **then**
    return $\phi_q(H)$
  end if
  $R \leftarrow \emptyset$
  for each path p touched by q **do**
    $h \leftarrow \text{Descend}(H, p)$
    if q asks for unknown product factors at this node **then**
      $h \leftarrow \text{Factorize}(h, C_{t,1}, \ldots, C_{t,F_t}, I)$
    end if
    $R \leftarrow R \cup \{\phi_q(h)\}$
  end for
  return $\text{Aggregate}_q(R)$
end procedure

5 Theory

The theorems we give here are relatively modest, but may be of significant practical value nonetheless. They show when the local-load story is valid; they do not claim universal lossless compression. The argument has four steps. We first bound the failure probability of a single cleanup step, in two forms: one for well-separated (incoherent) codebooks, and one that tracks the coherence of structurally correlated dictionaries such as recursively built subtree dictionaries. We then compose these bounds across the few cleanups that a fixed query family actually touches, which gives the central dimension law and shows that the requirement scales with local load, codebook separation, and supported queries rather than with raw object size. We then prove a matching lower bound, confirming that some restriction on the supported queries is unavoidable: a code that answered every leaf query would have to pay the full entropy cost. Finally we give a conditional margin theorem for recovering unknown product factors with the resonator, stated honestly as a basin-conditional result.

5.1 Cleanup capacity

Definition 2 (Coherence)**.** For a finite set of vectors $A \subset \mathbb{H}_D$ with $\|a\|^2 = D$ for all $a \in A$, define

$$\mu(A) = \max_{a \neq b \in A} \frac{|\langle a, b \rangle|}{D}. \tag{14}$$

For independent random atoms, μ concentrates at $O(\sqrt{\log |A|/D})$ and vanishes as D grows. For structurally correlated dictionaries—for instance two subtree codes sharing $k-1$ of k children—μ

contains a component that does *not* shrink with D; we call this the structural coherence, and denote by $\nu_t = \mu(K_t^{\mathrm{sub}})$ the coherence of the level-t subtree dictionary.

Lemma 1 (Deterministic cleanup margin)**.** *Let $z = a_\star + \eta$ where $a_\star \in C$, $\|a_\star\|^2 = D$, and cleanup is against C. Hard cleanup returns $a_\star$ if*

$$\langle a_\star, z\rangle - \max_{c\in C,\, c\neq a_\star} \langle c, z\rangle > 0. \tag{15}$$

In particular, if $\eta = \sum_{j=1}^{k-1} e_j$ and every pairwise normalized overlap among $C \cup \{e_j\}$ is at most μ, then cleanup succeeds whenever

$$1 > (2k-1)\,\mu. \tag{16}$$

Proof sketch. The target score is at least $D - (k-1)\mu D$ and any distractor score is at most $\mu D + (k-1)\mu D = k\mu D$. Positivity of the worst-case margin is $1 - (k-1)\mu > k\mu$, i.e. Eq. (16). The useful point is that deterministic success is a margin condition. □

Lemma 2 (Random single-step cleanup, incoherent codebook)**.** *Under Assumptions 1–3, suppose an unbinding step leaves one target atom plus $k-1$ crosstalk terms, that the crosstalk terms and the M codebook entries are mutually independent random atoms (equivalently, all reused content is decorrelated by distinct role products as in Assumption 3). Then for a constant $c > 0$ depending only on the atom distribution,*

$$\mathbb{P}[\textit{cleanup fails}] \le 2M \exp(-cD/k). \tag{17}$$

Proof sketch. The target score has mean D and fluctuations of order $\sqrt{kD}$. Each distractor score is sub-Gaussian with variance proxy $O(kD)$. A union bound over M distractors gives Eq. (17). □

Lemma 2 is the idealized law: reliability improves exponentially in the ratio D/k of dimension to local load, while the codebook size M enters only linearly inside the bound (logarithmically once one solves for the required D). But its independence hypothesis fails for exactly the dictionaries that hierarchical descent uses: two entries of K_t^{sub} that share most of their children are structurally correlated, and no amount of dimension makes them orthogonal. The next lemma gives the corrected law.

Lemma 3 (Coherence-limited single-step cleanup)**.** *Under Assumptions 1–3, suppose an unbinding step leaves one target codeword $a_\star \in C$ plus $k-1$ crosstalk terms whose randomness is independent of the identity of the confusable distractors, and let the codebook C (of size M) have coherence $\nu = \mu(C) < 1$. Then for a constant $c > 0$,*

$$\mathbb{P}[\textit{cleanup fails}] \le 2M \exp\big(-cD(1-\nu)/k\big). \tag{18}$$

Proof sketch. Fix a distractor c with $\langle a_\star, c\rangle = \nu' D$, $\nu' \le \nu$. Cleanup prefers $a_\star$ to c iff $\langle a_\star - c, z\rangle > 0$. The deterministic part is $\langle a_\star - c, a_\star\rangle = D(1-\nu')$. The noise part $\langle a_\star - c, \eta\rangle$ is a sum of $k-1$ sub-Gaussian terms, each with variance proxy proportional to $\|a_\star - c\|^2 = 2D(1-\nu')$, giving total variance proxy $O(kD(1-\nu'))$. The failure probability against this distractor is therefore at most $2\exp\big(-c\,D^2(1-\nu')^2/(kD(1-\nu'))\big) = 2\exp\big(-cD(1-\nu')/k\big)$, which is largest for the most coherent distractor $\nu' = \nu$. Union bound over M distractors. □

Remark 2 (Near-duplicate subtree dictionaries: the k^2 regime). For a recursively built subtree dictionary, two entries differing in a single one of k role-bound children have raw (pre-normalization) normalized overlap $\nu_{\mathrm{raw}} = 1 - \Theta(1/k)$. What survives normalization depends on the code family.

For linear (real-sum) entries, and for phasor codes whose per-component normalization is a phase rescaling with no sign-like collapse, the overlap remains $1-\Theta(1/k)$; substituting into Lemma 3 gives a per-step failure bound of order $M\exp(-cD/k^2)$, so distinguishing such near-duplicates reliably requires

$$D \gtrsim k^2 \log M. \tag{19}$$

For bipolar codes normalized by the sign function, however, quantization partially decorrelates near-duplicates. Treating each component of the two k-term sums as jointly Gaussian with correlation ν_{raw}, the post-sign correlation follows the arcsine law $\nu_{\text{sign}} = (2/\pi)\arcsin(\nu_{\text{raw}})$, and expanding $\arcsin(1-\epsilon) = \pi/2 - \sqrt{2\epsilon}\,(1+O(\epsilon))$ with $\epsilon = \Theta(1/k)$ gives

$$1 - \nu_{\text{sign}} = \Theta(k^{-1/2}), \qquad \text{hence} \qquad D \gtrsim k^{3/2} \log M \tag{20}$$

for sign-normalized MAP codes: quantization buys back a $\sqrt{k}$ factor for free. Both regimes are confirmed by the toy-scale study of Section 11. Either way the requirement exceeds $k \log M$, and the qualitative situation is the same. One further operational consequence, visible in the real-data example of Section 11.7, deserves statement as a design rule. When the query-time encoding is exactly consistent with the stored dictionary, cleanup is safe: a coherent neighbor cannot outscore an entry the query matches perfectly, so coherence risk stays latent. But when the query-time encoding is *noisy* relative to the stored dictionary—quantization jitter, stochastic normalization, representational drift, or any re-encoding mismatch—then below the coherence-aware budget of Eq. (21), snapping to a coherent dictionary can be net harmful relative to performing no cleanup at all, especially when accumulated crosstalk is mild (shallow depth, small load): the dominant error mode becomes near-duplicate confusion rather than crosstalk. The cleanup step on line 5 of Algorithm 2 should therefore be predicated on encoder–dictionary consistency, and treated as conditional on the dimension budget when that consistency cannot be guaranteed. This is not a pathology; it is the generic situation whenever the level dictionary contains entries that differ only in one slot. Three mitigations are available, and all three are measurable. (i) *Merge* confusable neighbors into a single effective entry, replacing M by the covering number $M_t^{\text{eff}}(\gamma)$ of Section 6.1; the price is that descent may then return the wrong member of a merged cluster, which is harmless for content shared by the cluster but corrupts any deeper query that descends into a branch where the members differ. The honest accounting is therefore per-path: for a depth-ℓ query, the relevant confusable set at each level is the set of neighbors that differ *on the queried path*, and errors are charged when the differing branch is entered. (ii) *Separate* the dictionary, by adding a per-entry summary or hash term inside Eq. (11) (the $\sigma_t \otimes S_\nu$ channel can carry a subtree-discriminating digest), or by training with the margin and confusability losses of Section 7, which directly push ν_t down for task-distinguished states. (iii) *Pay* the k^2 dimension cost where near-duplicate resolution genuinely matters. Which mitigation applies is a design decision that the experiments of Section 10 (E2b) are constructed to expose.

Theorem 1 (Fixed-family local query capacity)**.** *Consider a family of encoded factor trees with level subtree codebooks K_t^{sub} of sizes M_t^{sub}, coherences $\nu_t = \mu(K_t^{\text{sub}}) < 1$, local unbinding loads k_t, and maximum queried depth $\ell_{\max}$. Let Q_{sup} be a fixed set of at most $q_{\max}$ path queries. If*

$$D \;\geq\; \max_{t \leq \ell_{\max}} \frac{k_t}{c\,(1-\nu_t)} \log\!\left(\frac{2\, q_{\max} \sum_{s=1}^{\ell_{\max}} M_s^{\text{sub}}}{\varepsilon} \right), \tag{21}$$

then all descent cleanups required by all queries in Q_{sup} succeed with probability at least $1-\varepsilon$.

Proof sketch. At level t, Lemma 3 bounds the failure probability by $2M_t^{\mathrm{sub}} \exp(-cD(1-\nu_t)/k_t)$. Union bound over at most $q_{\max}$ queried paths and at most $\ell_{\max}$ levels. Rearranging gives Eq. (21). □

Remark 3 (Recovering the familiar law). For prototype codebooks made of independent random atoms (or dictionaries whose confusable neighbors have been merged at margin scale γ, with M_t^{sub} replaced by $M_t^{\mathrm{eff}}(\gamma)$ and ν_t by the post-merge coherence), $\nu_t = O(\sqrt{\log M_t^{\mathrm{sub}}/D})$ is asymptotically negligible and Eq. (21) reduces to the familiar $D \gtrsim k \log M$ law of Lemma 2. The coherence factor matters precisely for recursively built or otherwise structurally correlated dictionaries, where it produces the k^2 regime of Remark 2.

Remark 4 (When dimension is independent of raw size). Equation (21) has no direct N term. However, N can re-enter through $q_{\max}$, $\ell_{\max}$, k_t, M_t^{sub}, or—importantly—through ν_t: deeper trees at fixed branching contain more near-duplicate subtree pairs, so structural coherence can grow with depth even when nominal dictionary sizes do not. Thus the dimension is independent of raw object size only in the structured regime where supported queries are limited, depth grows slowly, local branching/load is bounded, level prototype dictionaries are reusable rather than object-specific archives, and dictionary coherence is controlled by merging, separation, or dimension.

Theorem 2 (All-path lower bound)**.** *Suppose a code supports simultaneous reconstruction of all N leaf labels drawn from an alphabet of size m with error probability bounded below a fixed constant. If each stored component carries at most q_{bits} bits after quantization, then*

$$D\, q_{\mathrm{bits}} = \Omega(N \log_2 m). \tag{22}$$

Consequently, no fixed-width code can be lossless for arbitrary N-leaf objects.

Proof sketch. There are m^N possible labelings. A quantized D-component code has at most $2^{Dq_{\mathrm{bits}}}$ states. Fano/counting arguments imply that faithful reconstruction of all labels requires enough states to distinguish an exponential number of labelings, giving the bound. □

5.2 Conditional resonator theorem

Let the target product be

$$y_\star = x_{1,\star} \otimes \cdots \otimes x_{F,\star}, \qquad x_{f,\star} \in C_f, \tag{23}$$

and let the observed vector be $H = y_\star + \eta$ after any relevant bundling and unbinding.

Definition 3 (Slot update margin)**.** At a resonator iteration, given projected estimates $y_g = \mathrm{Proj}_{\mathbb{H}}(\hat{x}_g)$ for $g \neq f$, define the slot-f evidence

$$z_f = H \otimes \Big(\bigotimes_{g \neq f} y_g^\dagger \Big). \tag{24}$$

The hard-cleanup margin for slot f is

$$\Delta_f = \langle x_{f,\star}, z_f \rangle - \max_{c \in C_f,\, c \neq x_{f,\star}} \langle c, z_f \rangle. \tag{25}$$

Theorem 3 (Conditional resonator success)**.** *Assume that during Algorithm 3, every slot update satisfies $\Delta_f \geq \gamma > 0$. Then hard cleanup at each update selects the true factor. For softmax cleanup*

with inverse temperature β, the probability mass assigned outside the true atom at that update is at most

$$(|C_f| - 1)\exp(-\beta\gamma). \tag{26}$$

If the margin condition holds for all slots for I iterations, final hard cleanup returns the true factor tuple.

Proof. The hard statement follows directly from the definition of Δ_f. For the softmax statement, every distractor logit is at least $\beta\gamma$ below the true logit, so the total distractor mass is bounded by a sum of $|C_f| - 1$ exponentials. Final hard cleanup is correct once every slot's true atom has the largest score. □

Definition 4 (Basin condition)**.** Say the resonator state at an iteration satisfies the *basin condition* (B) if, for every slot f, the evidence of Eq. (24) decomposes as

$$z_f = \alpha_f\, x_{f,\star} + \eta_f, \qquad \alpha_f \geq \alpha_0 > 0, \tag{27}$$

where η_f behaves as at most $k_f - 1$ effective independent crosstalk terms of unit atom scale relative to α_f (after the appropriate rescaling). Warm starts from partial cues, top-K beams, restarts, and external side information are the practical mechanisms for reaching (B); their cost is charged in Section 6.3.

Corollary 1 (Basin-conditional random margin)**.** *Condition on the basin condition (B) holding at every update of every iteration. Then the margin condition of Theorem 3 holds for all slots and all I iterations with probability at least*

$$1 - 2I \sum_{f=1}^{F} |C_f| \exp(-cD/k_f). \tag{28}$$

Remark 5 (Why the basin condition cannot be dropped). From the uniform initialization of Algorithm 3 (each $\hat{x}_g$ the normalized bundle of its whole codebook), the projected estimate y_g has correlation only $\Theta(|C_g|^{-1/2})$ with the true factor, so the slot-f evidence has target amplitude of order $\alpha_f = \Theta\big(\prod_{g\neq f} |C_g|^{-1/2}\big)$: exponentially small in the number of slots. The "target atom plus small independent crosstalk" model behind Corollary 1 therefore does *not* describe the early iterations from cold start, and no bound of the form (28) can be claimed unconditionally for Algorithm 3 as written. This matches what is known empirically: the operational capacity of resonator networks—the largest product-space size $\prod_f |C_f|$ that can be reliably factored from cold start—grows polynomially (not exponentially) in D [7]. The correct reading of the exponential-to-linear search claim is therefore: *conditional on basin entry* (via warm start, cue, beam, or restarts, all charged), each sweep costs $\sum_f |C_f|$ rather than $\prod_f |C_f|$, and the margin theory certifies the answer once found. Codebooks and factorizations should be trained so that realistic cues place the decoder inside the basin; this is one of the HDC-guided objectives of Section 7.

5.3 Task preservation for DNN latents

Recall from Definition 1 that each supported query q has a common answer space $\mathcal{Y}_q$, answer maps A_q^{full} and A_q^{fact}, and a loss d_q bounded by B_q. Let $A_q^{\text{HDC}}(H_x)$ denote the answer computed from the hypervector code by Algorithm 4: on the event that all descent cleanups and resonator calls succeed, $A_q^{\text{HDC}}(H_x) = A_q^{\text{fact}}(G_x)$ by construction. Let

$$B_{\max} = \sup_{q \in Q_{\text{sup}}} B_q. \tag{29}$$

Theorem 4 (DNN latent task preservation)**.** *Fix a supported query family Q_{sup} of size $q_{\max}$. Assume the DNN factorization defect is $\delta_{Q_{\mathrm{sup}}}$, level-codebook descent satisfies Theorem 1 with failure probability ε_d, and all resonator calls satisfy Corollary 1 (including its basin condition) with total failure probability ε_r. Then*

$$\sup_{q \in Q_{\mathrm{sup}}} \mathbb{E}_x \Big[d_q\big(A_q^{\mathrm{full}}(z(x)),\, A_q^{\mathrm{HDC}}(H_x)\big) \Big] \le \delta_{Q_{\mathrm{sup}}} + B_{\max}(\varepsilon_d + \varepsilon_r). \tag{30}$$

In particular, the recoverable HDC error can be driven down exponentially in $D(1-\nu)/k$ while the task-specific neural factorization defect remains as an irreducible floor.

Proof sketch. Decompose the event into successful local decoding/factorization and failure. On success, the code returns the quantized factor graph answer $A_q^{\mathrm{fact}}(G_x)$, whose expected discrepancy from the full answer is by definition at most $\delta_{Q_{\mathrm{sup}}}$. On failure, the loss is bounded by $B_{\max}$. Union bound descent and resonator failures. □

Remark 6 (On regularity of the losses). No Lipschitz assumption on d_q is needed for Theorem 4: the argument is a success/failure decomposition with bounded losses. A Lipschitz condition on d_q with respect to the answer would become necessary only if intermediate *soft*-cleanup readouts (the $\beta < \infty$ relaxation of Eq. (3)) were consumed directly as answers rather than hardened before readout, in which case the softmax mass bound (26) converts into an answer perturbation via the Lipschitz constant.

Remark 7 (Causal coding interpretation). If E_θ is trained by a causal or mechanistic objective, the factors in G_x may correspond to stable variables, mechanisms, objects, or interventions. The theorem does not assume that such factors are identifiable from passive raw data alone. It assumes only that a trained or weakly supervised encoder has produced a low-defect factor graph for the chosen query family.

5.4 Storage accounting

Let q_H be the number of bits per stored hypervector component. For bipolar codes $q_H = 1$ after sign quantization. For phasor or real-valued codes q_H is the chosen quantization precision. For a dataset of n items,

$$B_{\mathrm{code}} = nDq_H + B_{\mathrm{roles}} + B_{\mathrm{codebooks}} + B_{\mathrm{models}}. \tag{31}$$

If roles and codebooks are pseudorandomly generated from seeds, count the seeds and generator descriptions. If subtree prototypes or DNN quantizers are stored explicitly, count them. If an approximate-nearest-neighbor index is used for cleanup (Section 5.5), count the index. If the DNN encoder is required at query time, count its model storage and compute separately from the cached code. The per-item rate is small only when shared dictionaries and models are amortized over many items.

5.5 Speed and baseline accounting

Let $C_{\mathrm{HDC}}(q)$ be the cost of running query q on the code. Descent cleanup on line 5 of Algorithm 2 is against the subtree dictionary K_t^{sub}, so with naive dense cleanup a local query touching p paths of depth at most ℓ has cost

$$C_{\mathrm{HDC}}(q) = O\Big(p\,\ell\,D \max_t M_t^{\mathrm{sub}} \;+\; ID \sum_f |C_f|\Big). \tag{32}$$

Note the appearance of M_t^{sub}, the subtree dictionary size, rather than the node-prototype codebook size M_t of Eq. (7); the former can be much larger, and charging the smaller quantity would understate the cost of exactly the step that makes descent work. Approximate nearest-neighbor cleanup replaces the DM_t^{sub} factor by a sublinear search cost at the price of index storage (charged in Eq. (31)) and a measurable recall loss that must be folded into the cleanup failure probability. Structured codebooks, binary operations, and hardware parallelism further reduce constants.

Proposition 1 (Speed baselines)**.** *The meaningful speedup depends on the baseline.*

1. *Against an unindexed raw or latent scan costing $\Theta(N)$, a root summary or learned linear readout costing $O(D)$ has scan speedup*

$$\Sigma_{\text{scan}} = \Theta(N/D). \tag{33}$$

2. *Against a standard indexed tree path lookup costing $O(\ell)$, a local HDC path query costing Eq. (32) is not asymptotically faster. Its possible advantages are compression, batched computation, noise robustness, learned summaries, or hardware efficiency.*

3. *Against brute-force factor search over $\prod_f |C_f|$ tuples, the resonator update cost $O(ID\sum_f |C_f|)$ is exponential-to-linear in the factor codebook sizes, subject to the margin condition* and *the basin condition of Definition 4, with basin-entry overheads (restarts, beams, warm-start machinery) charged per Section 6.3.*

5.6 Compression-compute accounting

For a scan baseline and a single local query of fixed depth ℓ, define an amortized raw latent size of $N \log_2 m$ bits and an HDC item size of Dq_H bits. The compression ratio is

$$\rho_{\text{comp}} = \frac{Dq_H}{N \log_2 m}. \tag{34}$$

Using a local HDC cost proportional to $D\ell\, C_{\text{step}}$, the scan speedup is

$$\Sigma_{\text{loc,scan}} = \Theta\left(\frac{N}{D\ell\, C_{\text{step}}}\right), \tag{35}$$

and therefore

$$\rho_{\text{comp}}\, \Sigma_{\text{loc,scan}} = \Theta\left(\frac{q_H}{\log_2 m\ \ell\, C_{\text{step}}}\right). \tag{36}$$

For fixed depth, the product is independent of D and N; if $\ell = \log_b N$, it carries the expected logarithmic dependence on N. Shrinking D improves both nominal storage and HDC query cost, but only down to the fidelity threshold in Eq. (21). The real tradeoff is not compression versus speed; it is compression and speed versus distortion.

6 Scalability objections as design criteria

The theory above is useful only if its assumptions can be checked. The natural objections to hierarchical HDC—some of which we review here—are therefore best treated not as side issues but as measurable design criteria.

6.1 Effective rather than enumerative subtree dictionaries

A literal dictionary of every possible subtree at level t can be exponentially large. The theory should therefore not be interpreted as requiring enumeration of all generative compositions. Instead, the relevant quantity is an effective cleanup dictionary for the states that the task family actually distinguishes. Let $Z_t^{\mathrm{HD}} \subset \mathbb{H}_D$ be the image of the observed or task-relevant level-t latent states under the level encoder and quantizer (so that the covering is taken in the space where cleanup actually operates), and let

$$M_t^{\mathrm{eff}}(\gamma) = \mathcal{N}\big(Z_t^{\mathrm{HD}},\, d_{\mathrm{HDC}},\, \gamma\big) \tag{37}$$

be its covering number at cleanup margin scale γ, where d_{HDC} is the similarity metric used by cleanup. The local dimension condition can then be read as

$$D \gtrsim \max_t \frac{k_t}{1-\nu_t^{\mathrm{merged}}} \Big[\log M_t^{\mathrm{eff}}(\gamma) + \log(q_{\max}\ell_{\max}/\varepsilon) \Big], \tag{38}$$

where ν_t^{merged} is the residual coherence after merging confusable neighbors at scale γ. The method scales when the upstream learner collapses many raw configurations into a moderate number of reusable, well-separated latent states. It should fail when every subtree is essentially unique for the task. Merging is not free of semantics: as noted in Remark 2, a merged cluster answers correctly for content its members share and cannot answer for content on which they differ, so the merge scale γ must be chosen relative to the supported query family, and per-path confusability must be reported for deep queries.

For highly compositional nodes, a better strategy is factored cleanup rather than monolithic subtree cleanup:

$$u_\nu = c_{\nu,1} \otimes \cdots \otimes c_{\nu,F_t}, \qquad c_{\nu,f} \in C_{t,f}. \tag{39}$$

The resonator then searches over local factor dictionaries rather than over the explicit product dictionary. The practical advantage is not that the product space disappears, but that a high-margin product can be recovered by iterated slot-wise cleanup with cost proportional to $\sum_f |C_{t,f}|$ rather than $\prod_f |C_{t,f}|$, subject to the basin condition of Section 5.2.

6.2 Binding must be fixed or quasi-isometric

The proof of local cleanup assumes that binding and unbinding do not amplify crosstalk. This is reliable when binding is a fixed VSA operation such as Hadamard multiplication, XOR, phasor multiplication, permutation, or an orthogonal/unitary block map. If binding is learned freely by backpropagation, the system should not be expected to remain isometric.

A learnable binding family B_a should instead be constrained by a quasi-isometry condition on the relevant latent set:

$$\big|\langle B_a u, B_a v\rangle - \langle u, v\rangle\big| \le \eta_{\mathrm{bind}} D. \tag{40}$$

Then every cleanup theorem should be read with an effective margin

$$\gamma_{\mathrm{eff}} = \gamma - C_{\mathrm{path}}\eta_{\mathrm{bind}} D - C_{\mathrm{scale}}\eta_{\mathrm{scale}} D, \tag{41}$$

where C_{path} depends on the number of binding/unbinding operations in the query. In practical systems, the safest design is to let the DNN learn the factors and codebooks while keeping the HDC binding algebra fixed or strongly constrained.

6.3 Resonator search cost must be charged

A conditional margin theorem is honest, but it does not by itself guarantee that retrieval is cheap. If practical decoding uses R restarts, beam width K, I iterations, and dense cleanup over factor codebooks, then the local product search cost is

$$C_{\mathrm{res}} = O\Big(RKID \sum_{f=1}^{F} |C_f|\Big), \tag{42}$$

plus any recomposition scoring. The restart and beam factors R and K are precisely the price of basin entry (Remark 5); reporting them is therefore not optional bookkeeping but part of the claim itself. The resonator is computationally useful only when C_{res} is smaller than the avoided baseline: raw scan, dense attention over memory, full DNN recomputation, or brute-force enumeration of factor tuples. Approximate nearest-neighbor cleanup, structured codebooks, binary HDC hardware, and good warm starts can reduce constants, but they should be reported rather than hidden.

6.4 The factorization defect is a test, not a rescue clause

The task-specific defect δ_Q absorbs quantization error, missing nonlocal interactions, and holistic synergies. It should not be used to make the theorem vacuously true. It is the criterion that determines whether HDC is relevant to a task. The empirical protocol should always report

$$\Delta_{\mathrm{fact}} = L_{\mathrm{fact}} - L_{\mathrm{full}}, \tag{43}$$

$$\Delta_{\mathrm{HDC}} = L_{\mathrm{HDC}} - L_{\mathrm{fact}}. \tag{44}$$

If Δ_{fact} is large, the upstream factorization is inadequate. If Δ_{fact} is small but Δ_{HDC} is large, the HDC code is under-dimensioned, overloaded, geometrically distorted, coherent beyond its margin budget, or decoded with an insufficient search budget.

6.5 Admissibility checklist

For a target query family Q, a learned or hand-built factorization should be called HDC-admissible at dimension D only if the following quantities are small or favorable:

$$\delta_Q, \quad k_t, \quad \log M_t^{\mathrm{eff}}, \quad \nu_t, \quad M^{\mathrm{conf}}, \quad RKI, \quad \eta_{\mathrm{bind}} \tag{45}$$

and the cleanup margin exceeds the predicted crosstalk, structural coherence, and binding distortion. This checklist is also the bridge to learning: these are quantities the upstream system can be trained to improve.

7 HDC-guided factorization learning

The HDC layer is most plausible as a cache over learned or constructed factors, not as a direct replacement for a perception, language, or planning model. Its novel role is stronger than compression: it gives the upstream learner an operational target. The learner should not merely find a small latent. It should find a latent factor graph that is easy to bind, bundle, clean up, resonate, and query.

7.1 Representation families that can supply factors

Several existing neural representation families naturally supply pieces of the model.

Vector quantized latents. VQ encoders [9, 10] produce discrete latent codes. A multiscale VQ model supplies level codebooks K_t and reusable prototypes. The HDC code then bundles, binds, and queries these prototypes rather than storing every raw input coordinate.

Object-centric and slot latents. Slot or object-centric encoders [11] produce sets of latent object states. HDC roles can bind object identity, position, time, relation, occlusion status, and scale. The local load k_t becomes the number of objects or relations that compete at a given scale.

Sparse feature dictionaries for language. Sparse autoencoders over language-model activations [14, 15] provide overcomplete feature dictionaries. A document, sentence, or activation window can be represented as a sparse set of active features bound to token position, syntactic role, discourse role, layer, or head. The defect is low only for probes preserved by the dictionary; it is not a guarantee for full next-token generation.

Causal or mechanistic latent codes. A causal-coding encoder [13, 19] aims to discover variables corresponding to stable mechanisms, interventions, object states, or transition factors. If such a code has bounded local parents and reusable state prototypes, it directly satisfies the local-load assumptions. If the variables are not identifiable or the task depends on nonlocal synergies, this appears as a large δ_Q rather than as an HDC dimension problem.

Structured non-neural algorithms. The same idea applies beyond DNNs. A parser, theorem prover, program analyzer, database engine, causal-discovery procedure, planner, or symbolic reasoning system may already maintain a structured internal state. If that state has roles, local operations, reusable prototypes, or bounded-width factorization, the HDC layer can shape and cache it in the same way.

7.2 HDC-admissible factorization

The upstream learner should seek query-factorized sufficiency rather than global disentanglement [8, 12]. For a query family Q, the factor graph G_x is useful when

$$A_q^{\text{full}}(E_\theta(x)) \approx A_q^{\text{fact}}(G_x), \qquad q \in Q, \tag{46}$$

while each query touches only a few role-bound factors. A concise capacity proxy is

$$\mathcal{C}_{\text{HDC}} = \max_t \frac{k_t^{\text{eff}}}{1-\nu_t}\Big[\log M_t^{\text{conf}} + \log(q_{\max}\ell_{\max}/\varepsilon)\Big]. \tag{47}$$

The dimension requirement is roughly $D \gtrsim c^{-1}\mathcal{C}_{\text{HDC}}$. The HDC layer therefore asks the learner to reduce local load, reduce confusable alternatives, reduce structural coherence among task-distinguished states, and increase cleanup margins while preserving task sufficiency.

7.3 Measuring factorization error during learning

The basic diagnostic uses three readouts:

$$z(x) \to \hat{y}_{\text{full}}, \tag{48}$$

$$G_x \to \hat{y}_{\text{fact}}, \tag{49}$$

$$H_x \to \hat{y}_{\text{HDC}}. \tag{50}$$

Then

$$L_{\mathrm{HDC}} - L_{\mathrm{full}} = \Delta_{\mathrm{fact}} + \Delta_{\mathrm{HDC}} \tag{51}$$

with

$$\Delta_{\mathrm{fact}} = L_{\mathrm{fact}} - L_{\mathrm{full}}, \qquad \Delta_{\mathrm{HDC}} = L_{\mathrm{HDC}} - L_{\mathrm{fact}}. \tag{52}$$

This separates representation failure from HDC coding failure.

7.4 Local load loss

For a query q and example x, let $a_i(q, x) \geq 0$ be the relevance or attention weight assigned to factor i. The effective number of factors touched by the query is the participation ratio

$$k^{\mathrm{eff}}(q, x) = \frac{\big(\sum_i a_i(q, x)\big)^2}{\sum_i a_i(q, x)^2}. \tag{53}$$

Written this way the quantity is scale-invariant: multiplying all weights by a constant leaves k^{eff} unchanged, so the load regularizer below cannot be gamed by uniformly shrinking the weights (as the unnormalized form $1/\sum_i a_i^2$ could be). When the weights are normalized to sum to one, Eq. (53) reduces to the familiar $1/\sum_i a_i^2$. A load regularizer is

$$L_{\mathrm{load}} = \mathbb{E}_{x,q}\Big[\max\big(0,\, k^{\mathrm{eff}}(q, x) - k_{\mathrm{budget}}\big)^2\Big]. \tag{54}$$

This encourages the learner to organize information so that supported queries are local in the HDC sense.

7.5 Cleanup margin and confusability losses

For a latent vector z_ν assigned to prototype c^+, define

$$\Delta(z_\nu) = \langle z_\nu, c^+\rangle - \max_{c^- \neq c^+} \langle z_\nu, c^-\rangle. \tag{55}$$

The margin loss is

$$L_{\mathrm{margin}} = \Big[\gamma + \max_{c^- \neq c^+} \langle z_\nu, c^-\rangle - \langle z_\nu, c^+\rangle\Big]_+. \tag{56}$$

The margin target should scale with the predicted crosstalk. In normalized cosine form, a useful heuristic is

$$\gamma_t^{\cos} \approx C\sqrt{\frac{k_t \log M_t}{D}}. \tag{57}$$

The number of dangerous competitors is

$$M^{\mathrm{conf}}(z_\nu) = \#\big\{c : \langle z_\nu, c\rangle \geq \langle z_\nu, c^+\rangle - \gamma\big\}, \tag{58}$$

and can be penalized by $\mathbb{E}\log M^{\mathrm{conf}}(z_\nu)$. This is a better quantity than nominal codebook size when only a few states are actually confusable. A companion coherence penalty on the dictionary itself, $\big[\nu_t - \nu_{\mathrm{budget}}\big]_+$ computed over task-distinguished entry pairs, directly targets the near-duplicate regime of Remark 2.

7.6 Product recomposition and resonator identifiability

If a node latent is intended to factor as

$$z_\nu \approx c_{\nu,1} \otimes \cdots \otimes c_{\nu,F}, \tag{59}$$

then train not only for reconstruction but also for recoverability:

$$L_{\text{prod}} = \left\| z_\nu - \text{Normalize}(c_{\nu,1} \otimes \cdots \otimes c_{\nu,F}) \right\|^2, \tag{60}$$

while periodically measuring

$$p_{\text{res}} = \mathbb{P}\big[\textsc{Factorize}(z_\nu) \neq (c_{\nu,1}, \ldots, c_{\nu,F})\big], \tag{61}$$

under the intended query-time initialization (cold start, cued warm start, or beam), so that the measurement reflects the basin geometry actually available at deployment. A product code with low reconstruction error but high resonator failure rate is not HDC-admissible.

7.7 Role stability

A reusable content factor should remain stable when it appears in different roles. If $z_a \approx r_a \otimes c$ and $z_b \approx r_b \otimes c$, then

$$r_a^\dagger \otimes z_a \approx r_b^\dagger \otimes z_b. \tag{62}$$

The role-consistency loss on paired or augmented examples is

$$L_{\text{role}} = \left\| r_a^\dagger \otimes z_a - r_b^\dagger \otimes z_b \right\|^2. \tag{63}$$

For video this is object persistence; for language it is entity or predicate stability across syntactic and discourse roles; for causal coding it is mechanism stability across contexts.

7.8 Residual synergy test

Let $h_{\text{full}} = E_\theta(x)$ and let $h_{\text{fact}} = G_\omega(F_\psi(E_\theta(x)))$ be a reconstruction of the full latent from the factor graph. The residual

$$r = h_{\text{full}} - h_{\text{fact}} \tag{64}$$

should not contain task-relevant information. A residual probe $r \to \hat{y}_r$ estimates whether the factorization missed important synergy. If the residual probe performs well conditional on G_x, then δ_Q is high.

7.9 Intervention locality for causal coding

When paired examples differ by an intervention on factor j, the desired pattern is

$$c_i(x) \approx c_i(x') \;\; (i \neq j), \qquad c_j(x) \not\approx c_j(x'). \tag{65}$$

A simple intervention locality loss is

$$L_{\text{intervention}} = \sum_{i \neq j} d\big(c_i(x), c_i(x')\big) - \lambda\, d\big(c_j(x), c_j(x')\big). \tag{66}$$

This is exactly aligned with the HDC requirement that interventions change only a bounded number of role-bound factors.

7.10 Total HDC-guided objective

A representative training objective is

$$L = L_{\text{task}}^{\text{HDC}} + \lambda_\delta \Delta_{\text{fact}} + \lambda_{\text{margin}} L_{\text{margin}} + \lambda_{\text{load}} L_{\text{load}} + \lambda_{\text{prod}} L_{\text{prod}} + \lambda_{\text{role}} L_{\text{role}} + \lambda_{\text{cap}} \mathcal{C}_{\text{HDC}} + \lambda_{\text{res}} L_{\text{residual}}. \tag{67}$$

The point is not that every application must use every term. The point is that HDC supplies measurable pressure toward a particular kind of factorization: reusable discrete or nearly discrete factors such that each supported query depends on a small number of well-separated role-bound factors, with little task-relevant information left in the residual.

7.11 Live diagnostics

During training one should track

$$L_{\text{full}},\ L_{\text{fact}},\ L_{\text{HDC}},\ k^{\text{eff}},\ M^{\text{eff}},\ M^{\text{conf}},\ \nu_t,\ \Delta, \tag{68}$$

plus resonator success rate under the deployment initialization, resonator iterations, role-consistency error, and residual-probe accuracy. The learning process is moving in the HDC-admissible direction when

$$L_{\text{fact}} \approx L_{\text{full}}, \qquad L_{\text{HDC}} \approx L_{\text{fact}}, \tag{69}$$

while $k^{\text{eff}} \log M^{\text{conf}}/(1-\nu_t)$ decreases, cleanup margins increase, and residual-probe accuracy falls.

8 Resonant modular compositional episodic memory and world modeling

We now spell out a second major application of the general theory: episodic memory and world modeling in an architecture that uses modular compositional hypervectors as an intermediate representational regime. The key observation is simple but, in our view, important. If an episode itself has a factorized structure, then the episode should not be collapsed into a monolithic dense embedding before being stored. It should be compiled into a typed, role-filler, modular hypervector structure, and the resonator machinery should be allowed to operate on that structure.

This yields what we will call resonant hierarchical modular hypervectors, or R-HMH. The HMH side supplies the typed, ontology-aligned, named-slot address space; the resonator side supplies factor recovery, cleanup, and a quantitative pressure toward locally decodable low-load factors. The resulting memory item is not just a vector similar to an old experience. It is a compact cache of a structured episode whose parts can be separately queried, completed, swapped, aggregated, and consumed by neural or symbolic downstream systems.

8.1 Review: HMH and PRIMUS-style world modeling

Hierarchical Modular Hypervectors combine two ideas. First, type-based hierarchical named embeddings assign semantically meaningful slots to ontology types: for instance Demand Subject, Demand Object, Effect, Precondition, Location, Time, Value, Social Role, or Evidence. Second, modular compositional hypervectors use fixed-length hypervectors, role-filler binding, bundling, permutation, unbinding, cleanup, and approximate nearest-neighbor retrieval. The synthesis is an embedding whose slices or modules have human-interpretable functional meaning, while the

whole representation remains fixed-dimensional, noise-tolerant, algebraically compositional, and queryable by inverse binding [16, 17].

In a PRIMUS-style world model, HMH is not merely a compact storage format for episodes. It is a third representational regime between explicit symbolic atoms and dense neural tensors [18]. Symbolic atoms and rules carry crisp or uncertain declarative structure; dense vectors and tensors support perceptual prediction, control, and gradient learning; HMH stores structured associative keys over episodes, skills, affordances, social scripts, options, and mined templates. A map such as

$$D_{\mathrm{hmh}} : \mathbb{H}_D \to \mathbb{R}^d \tag{70}$$

then produces dense vectors usable by neural modules, while preserving, as much as possible, both the similarity geometry and the algebraic role-filler structure compiled into the HMH code.

This is exactly the setting in which resonator-HDC should be useful. A world model must often answer questions such as:

> Which prior interaction had the same social role pattern but a different actor? Which affordance trial had the same causal preconditions but a different object? Which episode had the same affective trajectory and repair outcome? Which skill was invoked in contexts with this goal, tool, and failure mode?

These are not whole-episode reconstruction queries. They are local, role-conditioned, factor-aware retrieval and completion queries.

8.2 A factorized episode model

Let an episode E be represented by a small typed factor graph

$$G_E = (S_E, C_E, A_E), \tag{71}$$

where S_E is a set of slot factors, C_E is a set of typed relations among slot factors, and A_E is a set of anchors to symbolic atoms, perceptual shards, logs, media, or motor traces. Typical slots include

$$\begin{aligned} S_E = \{&\text{actor, object, recipient, action, place, time, goal, plan, norm,} \\ &\text{affect, belief, tool, affordance, precondition, cause, outcome,} \\ &\text{reward, uncertainty, skill, script, evidence, perceptual trace}\}. \end{aligned}$$

The relation set C_E may include temporal order, causation, enablement, spatial adjacency, ownership, social obligation, repair, conflict, co-reference, part-whole structure, and counterfactual alternatives. Most of these relations are *directed*, and the encoding must preserve their orientation.

A basic R-HMH episode code has the form

$$\begin{aligned} H_E = \mathrm{Normalize}\Bigg(& r_{\mathrm{schema}} \otimes h_{\sigma(E)} \;\oplus \bigoplus_{s \in S_E} m_{\tau(s)} \otimes r_s \otimes h_{f_s} \\ & \oplus \bigoplus_{(i,j,\lambda) \in C_E} r_\lambda \otimes \big(r_i \otimes h_{f_i}\big) \otimes \rho\big(r_j \otimes h_{f_j}\big) \\ & \oplus \bigoplus_{a \in A_E} r_{\mathrm{anchor}(a)} \otimes h_a \Bigg). \end{aligned} \tag{72}$$

Here $h_{\sigma(E)}$ is a schema or event-type code, $m_{\tau(s)}$ is a module marker for the ontology type of slot s, r_s is the role vector for that slot, h_{f_s} is the filler code, r_λ is a relation-type vector for relation type

λ, and h_a is an anchor code pointing to external evidence or symbolic structure. In the relation term, the permutation ρ applied to the second (target) argument is essential and not decorative: under a commutative binding such as MAP, the symmetric product $r_\lambda \otimes r_i \otimes r_j \otimes h_{f_i} \otimes h_{f_j}$ is invariant under exchanging the two argument–filler pairs, so causes(interruption, embarrassment) and causes(embarrassment, interruption) would encode to the same vector, and temporal precedence would likewise lose its direction. Marking exactly one argument position with ρ (equivalently, with dedicated source/target roles as in the bridge encoding of Eq. (92) below) breaks this symmetry, in line with the ordering convention of Section 2. The same item therefore has three linked aspects: a symbolic graph, an HMH key, and optional perceptual or motor payloads. The HMH key does not replace the symbolic graph or the evidence store; it makes the episode addressable and partially decodable by vector-symbolic operations.

For example, a social interaction episode might bind teacher, child, interruption, embarrassment, repair, classroom, and later-positive-outcome into their named slots, while also binding directed relation factors such as causes(interruption, embarrassment) and repairs(apology, embarrassment). A Minecraft affordance trial might bind candidate tool, construction goal, adjacency pattern, redstone-like state, trial action, observed effect, and uncertainty into the same general format.

Note also that the same filler atom h_{f_i} may appear both in a slot term and inside one or more relation terms. By Assumption 3 these occurrences are decorrelated by their distinct role products, so they contribute independent-looking crosstalk rather than coherent interference; what must be watched is the total number of terms (slot load plus relation load), which enters the capacity criteria of Section 8.6 as $k_{\text{slot}} + k_{\text{rel}}$.

8.3 Resonant partial recall and slot completion

Given a cue Q, retrieval proceeds in two stages. An approximate-nearest- neighbor index first returns candidate episode codes by coarse similarity. Then role-masked unbinding and resonator cleanup recover or complete the factors relevant to the query. A cue such as

$$H_Q = r_{\text{social role}} \otimes h_{\text{teacher}} \;\oplus\; r_{\text{affect}} \otimes h_{\text{embarrassment}} \;\oplus\; r_{\text{outcome}} \otimes h_{\text{repair}} \tag{73}$$

should retrieve episodes sharing the social-affective-causal pattern even when specific names, utterances, or objects differ. Conversely, a cue such as

$$H_Q = r_{\text{affordance context}} \otimes h_{\text{wet adjacent circuit}} \;\oplus\; r_{\text{goal}} \otimes h_{\text{open door}} \tag{74}$$

should retrieve tool-use or construction episodes with structurally similar causal affordance patterns.

When a retrieved or queried factor is itself a product, resonator cleanup can be used inside the episode. Suppose a slot filler is intended to factor as

$$h_{f_s} \approx c_{s,1} \otimes c_{s,2} \otimes \cdots \otimes c_{s,F_s}, \qquad c_{s,j} \in C_{s,j}. \tag{75}$$

For a partial cue, the decoder iteratively forms evidence for one factor at a time,

$$z_{s,j} = \tilde{h}_{f_s} \otimes \Big(\bigotimes_{g \neq j} \text{Proj}_{\mathbb{H}}(\hat{c}_{s,g})^\dagger \Big), \qquad \hat{c}_{s,j} \leftarrow \text{cleanup}_{C_{s,j}}(z_{s,j}), \tag{76}$$

where $\tilde{h}_{f_s}$ is the slot estimate obtained by unbinding the appropriate role and module markers from H_E or from a retrieved candidate. Partial cues serve double duty here: any factor supplied by the cue acts as a warm start, placing the decoder inside or near the basin of Definition 4 for the

remaining unknown factors. This gives a natural mechanism for recovering unknown factors such as the likely goal, social role, affordance state, or missing outcome, provided the margin and basin conditions from Section 5 are satisfied.

The advantage is not that the memory can hallucinate arbitrary missing details. The advantage is that it can complete a bounded family of structured questions by searching factor codebooks rather than entire episodes. It can ask, for example, "which outcome factor makes this remembered affordance trial coherent with this goal and tool?" or "which social-script factor best completes this interaction pattern?"

8.4 Factor-aware neural judgment

A central motivation for PRIMUS-style use of HMH is that neural systems should be able to consume structured memories without losing the structure that made them useful. Let

$$z_E = D_{\mathrm{hmh}}(H_E) \tag{77}$$

be a dense representation of the episode. In the intended use, z_E is "dual-channel": one part preserves similarity geometry for neural matching, while another part preserves enough algebraic information to support role-filler sensitivity. A judgment model may then have the form

$$J_\phi(E, Q) = \Phi_\phi\big(D_{\mathrm{hmh}}(H_E),\, D_{\mathrm{hmh}}(H_Q),\, m_{\mathrm{slot}},\, m_{\mathrm{causal}},\, m_{\mathrm{temporal}}\big), \tag{78}$$

where the masks identify which slots, relations, or temporal windows are relevant to the current judgment. This lets a neural policy, critic, dialogue module, or world-model predictor condition on retrieved episodes while retaining access to who, what, where, why, with what causal role, and with what evidential support.

Counterfactual factor swaps are especially useful as training data. In a real-sum hypervector implementation one can form, schematically,

$$H_E[s := f'] = H_E - m_{\tau(s)} \otimes r_s \otimes h_{f_s} + m_{\tau(s)} \otimes r_s \otimes h_{f'}. \tag{79}$$

In sign or phasor systems the same operation is implemented by rebuilding the slot bundle from its factor record rather than by literal vector subtraction. The training question is then whether the judgment should be invariant or sensitive to the swap. Changing actor identity may be irrelevant for some affordance predictions but crucial for a social-trust prediction; changing causal relation, relation *orientation*, or outcome should often matter. Thus the memory representation directly supplies counterfactual supervision that respects the episode's factorization.

8.5 Episodic-semantic consolidation

R-HMH also gives a concrete mechanism for episodic-semantic braiding. Repeated retrieval of related episodes exposes common factor patterns:

$$\{H_{E_1}, \ldots, H_{E_n}\} \longrightarrow H_{\mathrm{template}} = \mathrm{Normalize}\Big(\sum_{i=1}^{n} w_i\, H_{E_i}^{\mathrm{aligned}}\Big), \tag{80}$$

where the alignments unbind and rebind corresponding slots so that different actors, objects, or contexts can be abstracted into role-level templates. A cluster of tool-use trials can become an affordance schema; a cluster of interaction repairs can become a social script; a cluster of repeated action sequences can become a skill or option template. The resulting template can be stored back as a symbolic rule, as an HMH associative key, and as a densified neural context vector.

The same process supplies a useful memory-management criterion. An episode should be retained when it has high future retrieval value, high evidence value, or high template-forming value. It should be consolidated when its idiosyncratic perceptual details are no longer needed for most supported queries but its factor pattern recurs. It should be re-encoded or pruned when ontology changes, cleanup margins fall, or residual probes show that important task information is no longer captured by the stored factors.

8.6 Capacity criteria for episodic memory

The capacity law from Section 5 now becomes an episodic-memory criterion. For a supported family of episodic queries Q_{epis}, define an episodic factorization defect

$$\delta^{\text{epis}}_{Q_{\text{epis}}} = \sup_{q \in Q_{\text{epis}}} \mathbb{E}_E \Big[d_q\big(A_q^{\text{full}}(\text{full episode state}),\, A_q^{\text{fact}}(G_E)\big) \Big]. \tag{81}$$

The R-HMH memory is useful only if this defect is small for the questions the agent actually asks. Its HDC coding error is then governed by effective slot load, relation load, confusable codebook size, dictionary coherence, cleanup margin, query depth, and the number of supported episodic queries.

A practical admissibility checklist is:

$$k_{\text{slot}}, \quad k_{\text{rel}}, \quad \log M^{\text{conf}}, \quad \nu, \quad \Delta_{\text{cleanup}}, \quad \delta^{\text{epis}}_{Q_{\text{epis}}}, \quad RKI, \quad \eta_{\text{bind}} \tag{82}$$

where k_{slot} is the number of simultaneously active slot factors, k_{rel} is the load due to typed relations among them, M^{conf} is the number of dangerous alternatives in the relevant cleanup dictionaries, ν is the coherence of those dictionaries, Δ_{cleanup} is the measured cleanup margin, RKI is the resonator search budget, and η_{bind} is binding distortion. If every episode requires a large holistic context vector to answer the task's questions, then the defect term is large and R-HMH is not the right representation by itself. If the factor graph preserves the supported questions but retrieval fails, then the HDC code is under-dimensioned, overloaded, too coherent, or decoded with insufficient search.

8.7 Operational workflow

A complete R-HMH episodic-memory loop can be summarized as follows.

1. **Ground and segment.** Perception, symbolic grounding, and local world-model inference create an episode graph G_E with slots, typed directed relations, and evidence anchors.

2. **Encode.** The graph is compiled into H_E by Eq. (72), while raw percepts and motor traces remain in specialized stores.

3. **Retrieve.** A symbolic or dense context forms an HMH query. AtomSpace or another symbolic store may pre-filter candidates; ANN search retrieves nearby HMH items.

4. **Decode and complete.** Role unbinding, module masks, and resonator cleanup recover the slots or product factors needed by the current query, with cue factors serving as warm starts.

5. **Densify and judge.** Retrieved items are mapped through D_{hmh} and used by neural policies, predictors, critics, or dialogue modules, with masks that preserve factor relevance.

6. **Consolidate.** Repeated high-value retrievals are abstracted into schemas, scripts, affordance templates, skills, or options, which are stored back in symbolic, HMH, and dense forms.

The result is a memory layer whose unit of storage is not a flat remembered scene, but a resonantly decodable factor graph cache. It can support partial recall, analogical retrieval, counterfactual judgment, evidence-grounded replay, and schema formation, while remaining compatible with the query-limited capacity theory developed above.

9 Instantiation in HBCML and ColBaC

The preceding sections are deliberately architecture-agnostic. They apply to any neural or algorithmic system that exposes reusable role-bound factors. The fit is particularly direct for Hierarchical Bayesian Causal Modular Learning (HBCML) [20] and Columnar Bayesian Causal Coding (ColBaC) [21, 22], because these architectures are already designed around sparse causal modules, internal certificates, local teachers, and reusable structure.

9.1 HBCML background

An HBCML system is a tuple

$$\mathcal{H} = (\mathcal{N}, \mathcal{C}, \Phi, \Psi, \mathcal{T}). \tag{83}$$

Here $\mathcal{N} = \{N_1, \dots, N_M\}$ is a family of structurally restricted component learners, $\mathcal{C}$ combines the outputs of the active components, Φ is the family of internal probabilistic states of the components, Ψ is a top-level probabilistic controller that selects supports and decides reuse, diversification, or recruitment, and $\mathcal{T}$ is a family of local counterfactual teachers. For context t, the controller selects a sparse support $S_t \subset \{1, \dots, M\}$ and the output has the schematic form

$$\hat{y}_t = \mathcal{C}\big((N_m(x_t; \theta_m, \phi_m))_{m \in S_t}; \Psi\big). \tag{84}$$

The HBCML design decomposes uncertainty across two structural scales. The top level faces a sparse combinatorial selection problem: which components should be active in this context? Inside each component, a second probabilistic process tracks whether internal material is reusable causal structure, semi-general stabilizing structure, task-local residue, or stale material due for recycling. The inside of a component emits certificates upward, and the controller uses those certificates to improve future selection.

The architecture is motivated by continual learning [25, 26, 23, 24]. If each task has an intended sparse support, gradients outside that support are small, cross-Hessian blocks between support and complement are small, and the combiner is Lipschitz, then the induced task update fields have small commutators. Sequential learning then approximates joint learning up to a controlled modularity error. HBCML is a constructive architecture designed to make these conditions hold by design, rather than hoping that a generic dense network discovers them on its own.

9.2 ColBaC background

ColBaC is a concrete columnar instance of HBCML. A ColBaC model contains a pool of columns. Each example activates a sparse subset of columns, plus any always-on shared columns. Each column contains typed microcolumns, typically

$$\text{K kernel-center processing}, \quad \text{L lateral local refinement}, \quad \text{B bridge or cross-scale context}. \tag{85}$$

Each microcolumn contains a protected hard kernel and adaptable shell tiers. The hard kernel is non-prunable and carries stable structure. Inner shells carry reusable abstraction, middle shells carry semi-general stabilizing material, and outer shells carry task-local exploratory residue. Promotion from outer to inner shells reflects consolidation; demotion outward reflects controlled forgetting.

The top-level controller can be trained in a teacher-first regime. At small scale, exact or exhaustive support search is possible. During learning, a local one-swap teacher can evaluate supports differing by one column and apply a swap only if the audited objective improves. A second teacher can audit internal shell swaps. This provides conservative local supervision for later heuristic or learned support-selection policies.

Internal certificates are central. A column can report shared-abstraction mass, specificity load, saturation pressure, demotion pressure, similarity signatures, probe influence, or other estimates of reuse utility. In Bayesian terms, conditioning the support controller on informative certificates can only improve the optimal mean-squared estimate of latent reuse utility. In engineering terms, the certificates tell the controller which columns are reusable, overloaded, stale, or close to the current context.

9.3 Why HDC fits ColBaC

ColBaC supplies the factorization that HDC needs: sparse supports, typed microcolumns, shell tiers, reusable motifs, and local teachers. HDC supplies what ColBaC needs: an algebraic representation of column identity, motif content, shell hygiene, support context, and certificates. The natural synthesis is

> ColBaC learns causal columns; HDC makes their reusable contents addressable, compositional, and auditable.

For column m, define a column hypervector

$$H_m(x) = \text{Normalize}\left(\bigoplus_{u\in\{K,L,B\}} \bigoplus_{s\in\{0,1,2,3\}} \bigoplus_{a\in A_m(x)} r_u \otimes r_s \otimes r_a \otimes c_{m,u,s,a}(x) \right). \tag{86}$$

Here u is the microcolumn type, $s = 0$ may denote the hard kernel and $s = 1, 2, 3$ the shells, a is a patch/time/context role, and $c_{m,u,s,a}$ is a cleaned-up motif atom or factored local product. In words, a column's code is the superposition of all its active motifs, with each motif triple-tagged by its microcolumn type, its shell tier, and its local role; this is exactly the hierarchical encoder of Eq. (11) specialized to the column's internal structure, so the same unbinding-and-cleanup machinery can read any tagged piece back out. Motifs reused across shells or patches are decorrelated by their distinct role products (Assumption 3); motifs that recur under the same role product should instead be deduplicated at encode time, or they will produce coherent crosstalk. The active support code is

$$H_{S_t}(x) = \text{Normalize}\Big(\bigoplus_{m\in S_t} r_m \otimes H_m(x) \Big). \tag{87}$$

This code can be used for support retrieval, context memory, certificate matching, and auditing.

9.4 HDC certificates

A scalar certificate vector can be replaced or augmented by a structured certificate hypervector

$$Z_m = r_{\text{shared}} \otimes z_{\text{shared},m} \ \oplus \ r_{\text{specific}} \otimes z_{\text{specific},m} \ \oplus \ r_{\text{saturation}} \otimes z_{\text{saturation},m} \\ \oplus \ r_{\text{demotion}} \otimes z_{\text{demotion},m} \ \oplus \ r_{\text{signature}} \otimes H_m. \tag{88}$$

The controller can then query structured certificate content by unbinding roles or by similarity against a context query vector. For example, it can ask for columns with high shared abstraction, low saturation, and a signature close to the current context.

A simple HDC support score is

$$\text{score}(m \mid x) = \langle Q(x), P_m \rangle + \lambda_Z \langle Q_Z(x), Z_m \rangle - \lambda_{\text{sat}} \, \text{saturation}(m) - \lambda_{\text{churn}} \, \mathbf{1}[m \notin S_{t-1}], \tag{89}$$

where $Q(x)$ is a context hypervector, P_m is the learned column prototype, and Z_m is the current certificate. The one-swap teacher audits the top retrieved supports and updates the prototypes and score model from the audit outcome.

9.5 HDC-guided shell hygiene

The HDC layer gives a precise rule for promotion and demotion. A motif should move inward when it is reused across contexts, has high cleanup margin, is stable under role unbinding, improves task loss through the factorized/HDC readout, and is not confusable with many other motifs. A motif should move outward or be recycled when reuse falls, margin falls, confusability rises, specificity load rises, or a residual probe shows that it no longer captures the intended task-relevant structure. Thus shell dynamics can use HDC quantities:

$$\text{promote}(c) \Leftarrow \text{reuse}(c) \uparrow, \ \Delta(c) \uparrow, \ M^{\text{conf}}(c) \downarrow, \ \Delta_{\text{fact}} \downarrow, \tag{90}$$

$$\text{demote}(c) \Leftarrow \text{reuse}(c) \downarrow, \ \Delta(c) \downarrow, \ M^{\text{conf}}(c) \uparrow, \ \text{specificity}(c) \uparrow . \tag{91}$$

The hard kernel should use fixed or nearly fixed HDC atoms. Inner shells should use high-margin cleanup. Middle shells can use soft cleanup or mixture codes. Outer shells should receive weak HDC regularization at first, to avoid premature discretization of exploratory material.

9.6 The bridge microcolumn

The B microcolumn is both powerful and dangerous. It carries cross-region, cross-scale, or cross-time context. In HDC terms, it is the channel for nonlocal bindings. If B becomes dense global attention, the local-load theorem no longer applies. The design rule is to force bridge content through sparse typed directed relations:

$$H_{B,m} = \bigoplus_{j=1}^{k_B} r_{\text{rel},j} \otimes r_{\text{src},j} \otimes r_{\text{tgt},j} \otimes c_{\text{rel},j}, \tag{92}$$

where the distinct source and target roles keep the relation oriented, exactly as the permutation does in Eq. (72). The bridge budget k_B should be measured and regularized just like any other local load.

9.7 Episodic support memory

A ColBaC-HDC memory item for context t can store the task, support, certificates, outcome, and audit result:

$$M_t = r_{\text{task}} \otimes c_t \ \oplus\ r_{\text{support}} \otimes \bigoplus_{m \in S_t} r_m \otimes H_m \\ \oplus\ r_{\text{certificate}} \otimes \bigoplus_{m \in S_t} r_m \otimes Z_m \ \oplus\ r_{\text{outcome}} \otimes y_t \ \oplus\ r_{\text{audit}} \otimes a_t. \tag{93}$$

Future contexts can retrieve similar memories by HDC similarity, warm-start the support controller, propose one-swap candidates, avoid overused columns, or suggest shell promotions and demotions.

9.8 Option memory for RL and agentic systems

In reinforcement learning, a column may encode a reusable option, skill, or world-model fragment rather than a perceptual motif. HDC gives a role-filler format for option memory:

$$H_{\text{option}} = r_{\text{pre}} \otimes H_{\text{pre}} \ \oplus\ r_{\text{act}} \otimes H_{\text{act}} \ \oplus\ r_{\text{eff}} \otimes H_{\text{eff}} \ \oplus\ r_{\text{value}} \otimes H_V \ \oplus\ r_{\text{uncertainty}} \otimes H_U. \tag{94}$$

The controller can query by current-state preconditions or desired effects to retrieve candidate options. This is a natural extension of HDC-guided factorization from perceptual memory to agentic action memory.

9.9 A ColBaC-HDC error decomposition

Let L_{ColBaC} be the task loss of the full ColBaC model, L_{fact} the loss when each active column is replaced by its factorized column graph, and $L_{\text{HDC-ColBaC}}$ the loss after HDC encoding, retrieval, and cleanup. Under HBCML adequacy errors $(\eta_{\text{app}}, \eta_g, \eta_h)$, HDC factorization defect δ_Q, binding distortion η_{bind}, local cleanup loads k_m, and dictionary coherences ν_m, a schematic bound is

$$L_{\text{HDC-ColBaC}} \le L_{\text{ColBaC}} + C_1\delta_Q + C_2 \sum_{m \in S_t} M_m \exp\big(-cD(1-\nu_m)/k_m\big) + C_3\eta_{\text{bind}}. \tag{95}$$

Similarly, the effective forgetting or update-commutator bound receives an additive representation term:

$$\big\|[X_{e_i}, X_{e_j}]\big\| \le C_H(\eta_h G_{\max} + \eta_g H_{\max} + \eta_{\text{app}}) + C_{\text{HDC}}\big(\delta_Q + e^{-cD(1-\nu)/k} + \eta_{\text{bind}}\big). \tag{96}$$

The constants and norms depend on the task model. The conceptual point is the three-way separation:

$$\text{continual-learning error} = \text{HBCML modularity error} + \text{factorization defect} + \text{HDC coding error}. \tag{97}$$

9.10 Why this is more general than ColBaC

ColBaC makes the HDC-guided idea especially direct because columns, shells, certificates, and one-swap teachers already provide the right hooks. But the principle is broader. Any algorithm with internal structure aligned to the data and tasks can use HDC requirements as a pressure on its own state: keep local interaction order small, keep role-bound factors stable, make codebooks well-separated and low-coherence, expose certificates, and ensure that task-relevant residuals are small.

10 Experimental program

10.1 Synthetic calibration

E1: single-step capacity. Bundle k random atoms, unbind a target, clean up against a codebook of size M, and fit the boundary $k \approx \alpha D / \log M$.

E2: descent with explicit subtree dictionaries. Generate bounded-branching trees with known K_t^{sub} whose entries are *well separated* (near-duplicates merged or excluded by construction). Vary N by increasing depth while keeping M_t^{sub}, ν_t, and k_t fixed. For a fixed query family, measure the D required to maintain a target error. The predicted curve is flat except for the explicit depth factor. Note that this flatness prediction is conditional on holding coherence fixed: if depth growth is allowed to introduce new near-duplicate subtree pairs, the required D will drift upward through ν_t even at fixed nominal dictionary size, which is a feature of the theory to be verified rather than a confound to be hidden.

E2b: near-duplicate stress test. Construct level dictionaries containing entry pairs that differ in exactly one of k role-bound children, so that $1 - \nu = \Theta(1/k)$ by design. Measure the D required to distinguish such pairs at fixed error as k grows. The prediction from Lemma 3 and Remark 2 is $D \propto k^2 \log M$ for linear or phasor entries and $D \propto k^{3/2} \log M$ for sign-normalized bipolar entries, against $D \propto k \log M$ for the well-separated control of E2; the cleanest diagnostic is the ratio $D^*_{\text{neardup}} / D^*_{\text{sep}}$, which isolates the coherence penalty $1/(1-\nu)$ from the shared load factor and should grow as $\Theta(k)$ or $\Theta(\sqrt{k})$ respectively (see Section 11 for a toy-scale confirmation of both regimes). Also measure the per-path error semantics of merged dictionaries: after merging the confusable pairs, verify that queries whose paths avoid the differing branch succeed at the merged (smaller) dictionary's predicted D, while queries entering the differing branch fail at a rate set by the merge.

E3: all-path negative control. At the same D, query every leaf. Global reconstruction should fail as N grows unless $Dq_H = \Omega(N \log m)$. This experiment verifies that the method is not secretly violating the entropy bound.

E4: resonator margin and basin. Measure convergence as a function of factor codebook sizes, local load, initialization (cold start versus cued warm start versus beam), damping, projection, and margin. Report recomposition scores for successes and spurious fixed points, and estimate the operational capacity curve (largest reliably factorable product space versus D) for comparison against [7].

E5: ablations. Remove cleanup, remove ordered roles (including the argument-position permutation on directed relations, which should collapse relation orientation), remove level codebooks, replace structured latents by random nonfactorized latents, and run a nondecomposable task such as global parity. Each ablation should destroy the corresponding theorem condition.

10.2 DNN latent experiments

Video. Use a pretrained or trained video encoder that yields object slots, VQ tokens, or latent events. Build a factor graph over object identity, position, velocity, relations, and time. Test object tracking, event retrieval, action classification, and anomaly detection. Plot task accuracy versus D,

cleanup failure versus local load and measured coherence, and the full-DNN-minus-HDC accuracy gap as an empirical estimate of δ_Q.

Images. Use multiscale VQ tokens, patches, scene graphs, or slots. Test object/property queries, localized retrieval, and scene classification. Include a texture-heavy or global-style task as a high-defect control.

Language. Use token/span hierarchies, parse roles, retrieval chunks, or sparse-autoencoder features from a language model. Test entity retrieval, document routing, attribution, syntax/semantic probes, and structured QA over cached features. Do not claim faithful open-ended generation unless the defect is measured for that task.

Baselines. Compare against the full DNN latent readout, a compressed latent baseline such as PCA or product quantization, an indexed tree or ANN index, a raw latent scan, a nonhierarchical HDC bundle, and a no-cleanup HDC bundle. The HDC method should win only where local load is bounded and the query family matches the factorized structure.

10.3 HDC-guided learning experiments

Train an encoder with and without the HDC-guided losses of Section 7. Compare the curves for Δ_{fact}, Δ_{HDC}, k^{eff}, M^{conf}, ν_t, cleanup margin, resonator success under the deployment initialization, and residual-probe accuracy. The prediction is not merely that HDC-guided training improves final accuracy. The sharper prediction is that it moves these intermediate diagnostics in the capacity-favorable direction.

A useful ablation sequence is:

1. train the full DNN task model;
2. add factorization but no HDC losses;
3. add margin, load, and coherence losses;
4. add role-consistency and product-recomposition losses;
5. add the actual HDC encode/decode path in the task loop.

This separates whether failures come from the factor graph, the HDC geometry, or the task head.

10.4 R-HMH episodic memory and world-modeling experiments

A direct experiment for Section 8 is to compare flat episodic embeddings, ordinary HMH episodic keys, and R-HMH episodic keys with resonator cleanup. Useful task families include partial cue completion, factor-specific retrieval, directed-relation queries (does the memory distinguish causes(a, b) from causes(b, a), and before(a, b) from before(b, a)?), counterfactual factor swaps, affordance-trial recall, social-script retrieval, and schema consolidation. Report whole-episode ANN accuracy separately from slot recovery accuracy, because a system can retrieve a roughly similar episode while still failing to recover the causal or social factor that matters for the query.

A minimal benchmark should track

$$\Delta_{\text{fact}}^{\text{epis}},\ \Delta_{\text{HDC}}^{\text{epis}},\ k_{\text{slot}},\ k_{\text{rel}},\ M^{\text{conf}},\ \nu,\ \Delta_{\text{cleanup}},\ p_{\text{res}}, \tag{98}$$

plus downstream judgment accuracy for neural models trained on raw dense embeddings versus $D_{\mathrm{hmh}}(H_E)$. The expected win is not perfect replay of all perceptual detail, but improved factor-aware recall and better downstream judgment when the task depends on the structure of the episode.

10.5 ColBaC-HDC experiments

For ColBaC/HBCML, compare four systems:

$$\begin{aligned}&\text{A: ColBaC baseline,}\\&\text{B: ColBaC + HDC certificates,}\\&\text{C: ColBaC + HDC factorization losses,}\\&\text{D: ColBaC + HDC certificates + factorization losses + episodic support memory.}\end{aligned} \tag{99}$$

The most likely early win is not raw benchmark accuracy but improved selector geometry: lower selector regret, better one-swap warm starts, less support churn, better prediction of audited support quality, and more stable shell promotion. Report

$$L_{\text{full ColBaC}}, \quad L_{\text{factorized ColBaC}}, \quad L_{\text{HDC-ColBaC}}, \tag{100}$$

plus support rank, one-swap recovery, support entropy, support churn, promotion precision, cross-column HDC overlap (a direct coherence measurement), certificate-predicted audit gain, and forgetting. If HDC certificates are useful, they should improve retrieval of good support neighborhoods before the one-swap teacher acts.

11 Toy-scale numerical validation and a real-data example

Before the full experimental program of Section 10, it is worth knowing whether the theory survives contact with even a minimal implementation. This section reports four deliberately small numerical studies, implemented in a few hundred lines of NumPy. The first exercises the paper's core machinery end to end on synthetic data: hierarchical encoding, level-dictionary descent, coherence-limited cleanup, resonator factorization with and without basin entry, and directed-relation encoding. The second (Section 11.7) runs the full pipeline of Eq. (1) on real data—quadtree-factorized handwritten digit scans with k-means codebooks—so that the level dictionaries, their coherence, and the factorization defect are learned and measured rather than constructed. The third (Section 11.8) repeats the pipeline in the temporal domain, on real spoken-digit audio, where the ordering convention of Section 2 becomes empirically load-bearing and two distinct query families are served by one code. The fourth (Section 11.9) turns from analysis to synthesis, asking whether the same machinery can reduce the per-frame cost of streaming autoregressive generation over a residual-quantized audio codec—the setting in which the compute claims of Section 5.5 would actually be cashed. None of these studies is a benchmark; the scale is deliberately toy. Their purpose is falsification pressure on the specific quantitative claims of Section 5, and in two cases (the arcsine refinement of Remark 2, and the conditional-cleanup lesson of Section 11.7) they sharpened the theory rather than merely confirming it.

11.1 Implementation

The implementation uses the bipolar MAP model. Atoms are i.i.d. uniform vectors in $\{-1,+1\}^D$; binding is the elementwise (Hadamard) product, under which every bipolar vector is its own inverse;

bundling is integer summation; normalization is the sign function with a fixed per-dimension tie-breaking sign convention, so that encoding is a deterministic function of its input, as the theory assumes (one experiment below deliberately randomizes the tie-breaking as a controlled model of encoder noise); the permutation ρ is a cyclic shift; hard cleanup is a matrix-vector product against the codebook followed by argmax. The hierarchical encoder implements Eq. (11) without the summary channel; descent implements Algorithm 2 with explicit subtree dictionaries; the resonator implements Algorithm 3 with the standard project-onto-codebook update $\hat{x}_f \leftarrow \mathrm{sign}\big(C_f C_f^\top z_f\big)$ and convergence detection. Five experiments were run, labeled to match Section 10.

11.2 T1: single-step cleanup capacity

Bundle k role-bound atoms, unbind one role, and clean up against a codebook of $M = 256$ independent atoms. Accuracy over 300 trials:

k	D=256	D=512	D=1024	D=2048	D=4096
4	1.000	1.000	1.000	1.000	1.000
16	0.637	0.950	1.000	1.000	1.000
64	0.117	0.307	0.657	0.910	1.000

The accuracy frontier tracks $D \propto k$ at fixed M, as Lemma 2 predicts: $k = 4$ is already perfect at $D = 256$, $k = 16$ crosses 95% near $D = 512$, and $k = 64$ requires roughly $D = 4096$.

11.3 T2: hierarchical descent and the role of cleanup

Depth-3 trees with branching $b = 4$, a leaf vocabulary of 64 atoms, and sampled level subtree dictionaries of size $M^{\mathrm{sub}} = 128$ were encoded by Eq. (11), and random root-to-leaf paths were queried by Algorithm 2, with and without the level-dictionary cleanup on line 5. Leaf-recovery accuracy over 150 trials:

D	with cleanup	without cleanup
512	1.000	0.180
1024	1.000	0.320
2048	1.000	0.487
4096	1.000	0.853

With cleanup, descent is already perfect at $D = 512$. Without it, sibling crosstalk accumulates multiplicatively across levels exactly as the theory warns, and even an eightfold larger dimension does not recover reliability. Level-dictionary cleanup, not dimension, is what makes descent work.

11.4 T2b: near-duplicate scaling and an arcsine-law refinement

The near-duplicate stress test of E2b was run at dictionary size $M = 32$: for each load $k \in \{4, 8, 16, 32\}$, a level dictionary was built whose first two entries differ in exactly one of k role-bound children, descent-style noise of $k - 1$ sibling atoms was added to the first entry, and the minimal dimension D^* achieving 95% cleanup accuracy was found by grid search (400 queries per grid point, dictionaries redrawn throughout). A well-separated control used fully independent entries. Two variants were run: dictionary entries normalized by sign (the paper's default), and entries kept as raw linear sums with cosine cleanup.

	sign-normalized entries			linear (unnormalized) entries		
k	D^*_{sep}	D^*_{nd}	ratio	D^*_{sep}	D^*_{nd}	ratio
4	48	64	1.33	24	24	1.00
8	96	128	1.33	24	48	2.00
16	256	384	1.50	24	96	4.00
32	512	1024	2.00	24	192	8.00

The well-separated condition scales as $D^* \sim k^{1.14}$ (log-log fit over $k = 4$ to 32), matching the $k \log M$ law. The informative diagnostic is the ratio column, which isolates the coherence penalty $1/(1-\nu)$ from the load factor shared by both conditions. For linear entries the ratio is exactly $k/4$: perfectly linear in k, confirming the $1 - \nu = \Theta(1/k)$ overlap and hence the $k^2 \log M$ law of Eq. (19). For sign-normalized entries the ratio grows only like $\Theta(\sqrt{k})$ ($1.33 \to 2.00$ over an $8\times$ range of k), which is what first motivated the arcsine-law analysis now recorded in Remark 2: sign quantization maps raw overlap $1 - \Theta(1/k)$ to $1 - \Theta(k^{-1/2})$, softening the near-duplicate law to $k^{3/2} \log M$ for bipolar sign codes. This is the one place where the toy run sharpened the theory rather than merely confirming it: the natural first conjecture—that the $\Theta(1/k)$ overlap survives normalization up to constants—is what the data show to be too pessimistic for sign codes, and the arcsine-law analysis of Remark 2 was formulated in response. It also yields a small design lesson: where near-duplicate resolution is the binding constraint, sign-quantized bundles are preferable to linear or phasor bundles, independent of their storage advantages. (In the linear condition the well-separated D^* is flat in k because the unnormalized target's norm grows as $\sqrt{kD}$ while the sibling crosstalk does not; the two conditions there differ only in the presence of the near-duplicate, which is exactly why the ratio is the meaningful column.)

11.5 T4: resonator operational capacity and basin entry

Three-factor products $H = x_1 \otimes x_2 \otimes x_3$ with equal codebook sizes $|C|$ were decoded at $D = 1024$ from the uniform cold-start initialization of Algorithm 3, and, for comparison, from a warm start in which one of the three factors is supplied as a cue and unbound before decoding the remaining two. Accuracy over 100 trials:

$\lvert C\rvert$ per slot	product space	cold start	warm start (1 cued)
8	512	0.99	1.00
16	4,096	0.83	1.00
32	32,768	0.61	1.00
64	262,144	0.29	1.00
128	2,097,152	0.01	0.98

Cold-start accuracy collapses as the product space grows past the operational capacity at this dimension—from 99% at 512 tuples to 1% at two million—exactly the polynomially bounded capacity reported by [7] and assumed by Remark 5. Supplying a single cue restores essentially perfect accuracy at every size tested. This is the basin-conditional Corollary 1 in one table: the exponential-to-linear search win is real, but it is purchased by basin entry, and partial cues are a cheap and effective entry mechanism.

11.6 T5: directed relations

Finally, the directed-relation encodings were compared directly. Under the naive symmetric encoding $r_\lambda \otimes r_i \otimes r_j \otimes h_a \otimes h_b$, the codes for causes$(a, b)$ and causes(b, a) have cosine similarity $+1.000$:

orientation is completely lost, as commutativity dictates. Under the encoding of Eq. (72), with the permutation applied to the target argument, the two orientations have cosine -0.003: statistically orthogonal, and cleanly distinguishable by any downstream readout.

11.7 A real-data example: quadtree-factorized handwritten digits

The synthetic study controls every quantity by construction. A complementary question is what the paper's objects look like when they are *learned* from real data: how large is an effective subtree dictionary in practice, how coherent is it, where does the factorization defect land for a real task, and does descent behave as the theory predicts. To answer this at small scale, the full pipeline of Eq. (1) was run on the classical handwritten digits corpus of 1,797 real 8×8 grayscale scans (as distributed with scikit-learn), split into 1,397 training and 400 test images.

The factor graph F_ψ is a quadtree of depth 2 and branching 4: each image decomposes into four 4×4 quadrants, and each quadrant into four 2×2 patches. The leaf quantizer is a k-means codebook of $M_2 = 16$ prototypes fitted to the 22,352 real training patches (quantization mse 3.5 on pixel values in $[0, 16]$). The mid-level subtree dictionary K_1^{sub} is exactly the effective dictionary of Section 6.1 realized empirically: the set of quadrant states (distinct 4-tuples of leaf tokens) actually observed in training. Encoding is the bipolar MAP instantiation of Eq. (11) without the summary channel, and descent is Algorithm 2.

R1: the learned dictionary is small but pervasively coherent. Of $16^4 = 65{,}536$ formally possible quadrant states, only $M_1^{\text{sub}} = 972$ occur in training (1.5%), and they cover 92.4% of test quadrants. This is the effective-dictionary premise of Section 6.1 confirmed on real data: the task-relevant state space is orders of magnitude smaller than the formal product space. The coherence picture is equally clear and less comfortable: the dictionary contains 6,559 structural near-duplicate pairs (entries differing in exactly one of four children), and 99.6% of entries (968 of 972) have at least one near-duplicate neighbor, with sampled coherence $\nu_1 \approx 0.65$–0.72 (a figure that includes a floor contributed by the shared tie-breaking convention as well as the structural overlap). On real learned dictionaries, near-duplicates are not an edge case; they are the generic condition, which is why Remark 2 is stated as a design constraint rather than a pathology.

R2: descent, and when cleanup helps. Leaf-token recovery by descent from the single root hypervector was measured on 600 random (test image, quadrant, patch) queries (restricted to the 92.4% of quadrants covered by the training dictionary), with and without the mid-level cleanup of Algorithm 2 line 5. Two encoder conditions were run. In the *consistent* condition, the encoder is the deterministic function described above, so a covered test quadrant encodes to exactly its dictionary entry. In the *noisy* condition, sign ties are instead re-randomized independently at every encoding call—a controlled, implementation-level model of encoder noise, standing in for the quantization jitter, stochastic normalization, or representational drift that any deployed system accumulates between the time a dictionary is built and the time it is queried (with four-term bipolar bundles, roughly 37% of components tie, so the injected noise is substantial):

encoder	D	with cleanup	without cleanup	mid-level errors
consistent	1024	1.000	0.984	none
consistent	4096	1.000	1.000	none
consistent	16384	1.000	1.000	none
noisy	1024	0.967	0.993	93, of which 95% near-duplicate
noisy	4096	1.000	1.000	none

Three things are visible. First, at adequate dimension everything works under both conditions: $D = 4096$ gives perfect recovery every way, consistent with the coherence-aware budget of Eq. (21), for which $k/(1-\nu_1) \cdot \log M_1^{\text{sub}}$ is a few hundred here. Second, with a consistent encoder, cleanup is beneficial at every tested dimension, including below budget: it corrects the crosstalk-induced errors that raw descent makes at $D = 1024$ (100% versus 98.4%), and coherence does no harm because the query snaps to a dictionary entry it matches exactly. Third, and most instructive, under encoder noise *below* the budget the preference inverts: cleanup (96.7%) becomes worse than no cleanup at all (99.3%), and the error autopsy shows that 95% of the mid-level cleanup errors select a near-duplicate neighbor of the true state, which then corrupts the leaf query exactly when it descends into the differing branch—the per-path error semantics of Remark 2 observed live. The mechanism is a tradeoff the theory makes legible: cleanup buys crosstalk suppression at the price of coherence risk, and encoder noise is what converts that risk from latent to realized, by pushing the query off its own dictionary entry and into a near-duplicate's basin. (For contrast, in the synthetic depth-3 study above, with compounding crosstalk and an incoherent dictionary, cleanup was indispensable: 100% versus 18%.) The design rule that falls out is stated in Remark 2: level-dictionary cleanup is safe when the query-time encoding is consistent with the stored dictionary, but under encoder noise it should be treated as conditional on the coherence-aware dimension budget being met.

R3: the error decomposition on a real task. Digit classification with a linear readout gives the three-way comparison of Section 7: raw pixels (L_{full} readout) 95.8%; factor-graph tokens (L_{fact}) 94.5%, so $\Delta_{\text{fact}} = 1.3$ points—the quadtree-plus-VQ factorization loses almost none of the class information; root hypervectors (L_{HDC}, consistent encoder) 94.0% at $D = 1024$ and 95.3% at $D = 4096$, so Δ_{HDC} is 0.5 points and -0.8 points respectively: at adequate dimension the coding stage adds essentially no task loss, and the whole (small) gap to the full readout is the factorization defect. Under the noisy-encoder condition of R2 the same readout drops to 89.5%–90.7%, i.e. encoder noise costs 4–5 points through the same mechanism that inverts the cleanup preference. The protocol does what it is for: it localizes error between the factorization, the code, and the encoder's consistency with it, and says where effort should go.

R4: role-bound content queries. Finally, inverse queries of the form "which of the 16 patch positions holds leaf token c?" were answered directly from the root code by scoring it against the 16 role-product probes, without any descent. Over 300 random (image, token) probes, position detection achieves AUROC 1.000 (under the consistent encoder; $n = 165$ informative probes; chance 0.5): role-bound content is linearly visible in the single root vector, as the ordering convention of Section 2 intends.

11.8 A third study: spoken digits and temporal order

The image example has spatial structure but no intrinsic direction: a quadrant order is a convention. Audio adds the ingredient that motivated the ordering convention of Section 2 in the first place—time has a direction, and a memory that cannot tell an event sequence from its reversal cannot support temporal-order or causal-order queries at all. The same pipeline was therefore run on the Free Spoken Digit Dataset: 3,000 real 8 kHz recordings of the digits zero through nine by six speakers, split 2,400/600. Features are 16-band log-mel frames (32 ms windows, 16 ms hop, 48 frames per utterance after center-cropping or padding, per-utterance mean-variance normalization). The factor graph is a temporal hierarchy: utterance $\rightarrow$ four temporal segments $\rightarrow$ four leaves each, where a leaf is the average of three consecutive frames. The leaf quantizer is a k-means codebook of

$M_2 = 32$ prototypes fitted to the 38,400 real training leaves, and the mid-level dictionary is again the set of segment states (distinct 4-tuples of leaf tokens) observed in training.

A1: the dictionary picture replicates, with lower coverage. Of $32^4 \approx 1.05 \times 10^6$ formally possible segment states, only 3,854 occur in training (0.37%); they contain 11,763 near-duplicate pairs, and 93.6% of entries have at least one near-duplicate neighbor. Both halves of the vision finding thus replicate in a second modality: effective dictionaries are a vanishing fraction of the formal product space, and coherence within them is pervasive. The notable difference is coverage: the training dictionary covers only 71.1% of test segments, against 92.4% for the digit images. Real speech is simply more variable at this granularity, and the shortfall is the honest cost of an enumerated effective dictionary; the remedies are those of Section 6.1—more data, coarser merging at the price of per-path confusability, or factored cleanup in place of enumeration.

A2: descent replicates. With the consistent (deterministic) encoder, leaf-token recovery by descent from the root code over covered segments is perfect with cleanup at every tested dimension (D = 1024, 4096, 16384) and marginally worse without it at $D = 1024$ (99.3%), with sampled dictionary coherence $\nu_1 \approx 0.65$–0.68. The vision conclusions carry over unchanged.

A3: two query families from one code. The supported-query story of Section 3 says one code should serve whatever query families it was budgeted for. Here two natural families coexist: what was said (digit, 10-way) and who said it (speaker, 6-way). Linear readouts give:

query	raw log-mel (L_{full})	tokens (L_{fact})	root code (L_{HDC}, D=4096)
digit (10-way)	0.813	0.768	0.770
speaker (6-way)	0.768	0.727	0.745

Two observations. First, Δ_{fact} is 4–5 points for both queries—noticeably larger than for the digit images—while Δ_{HDC} is within noise of zero (the root-code readout matches or slightly exceeds the token readout, which is possible because the code is a different featurization of the same token information). The decomposition therefore localizes essentially all of the loss in the factorization stage: sixteen averaged, vector-quantized leaves discard real acoustic detail, and no amount of HDC dimension will recover it. This is the defect term doing its diagnostic job on real data—the factor graph, not the code, is what needs improving here. Second, both query families remain decodable from the same fixed-width root vector, phoneme-like content and speaker identity alike, which is the multi-query premise of the capacity law exercised on real signals.

A4: ordered roles are what preserve the arrow of time. Finally, the direct test. Each utterance's token array was time-reversed (segment order and leaf order within segments), both were encoded, and the cosine between forward and reversed codes was measured under two encoders: the standard one with ordered temporal roles, and a control in which all sibling roles at both levels are identical (role-free bundling), so that binding commutativity is left to do whatever it does. With role-free bundling, the forward and reversed codes are *identical*: mean cosine +1.000, exactly, because reversal permutes summands inside each bundle and a commutative superposition of the same multiset is the same vector. Such a code provably cannot answer any temporal-order query. With ordered roles, the mean forward-reversed cosine is +0.360, against a measured baseline of +0.298 between codes of entirely unrelated utterances (the baseline is nonzero because real utterances share common tokens such as silence and steady vowels, and because the fixed

tie-breaking convention contributes a small shared floor). An utterance's reversal is thus rendered almost exactly as dissimilar as a different utterance altogether. This is the temporal analog of the directed-relation requirement of Section 8.2, measured on real speech: orientation—of time, causation, or precedence—is not preserved by superposition; it is preserved only by the ordered roles or argument-position permutations that the encoding must supply, and the cost of omitting them is not graceful degradation but total, exact confusion of a sequence with its mirror.

11.9 A fourth study: streaming generation over a residual-quantized codec

The studies above treat the hypervector code as a memory to be queried. The compute model of Section 5.5, however, makes claims about *generation*: resonator decoding replaces brute-force factor search, and role-bound summaries replace dense attention over stored context. Modern neural audio generation is a natural place to test both, because it already operates on a factored discrete representation. A residual vector-quantized (RVQ) codec represents each frame as a sum of K codewords drawn from K successive codebooks, and an autoregressive model predicts those tokens frame by frame at a fixed rate. Two costs dominate the real-time budget: the output interface, which must emit K tokens per frame (often serialized, adding structural latency), and the history interface, whose attention cost grows with the length of the stored context.

To test both at toy scale, a three-stage RVQ ($K = 3$ codebooks of 16 codewords) was fitted to the log-mel frames of the spoken-digit corpus of Section 11.8, giving 144,000 real frames with residual reconstruction error 0.088 against 0.196 for the first stage alone, so the residual structure is genuine rather than nominal. Consecutive frames share a token with probability 0.77, 0.64, and 0.61 at the three stages respectively: this temporal persistence is the resource that Definition 4 calls basin entry, available for free in streaming settings. Because the object under test is the *interface* and not the predictor, all conditions below use linear predictors on identical inputs (a four-frame log-mel window plus the previous frame's token identities), so that differences are attributable to the output or context representation alone.

G1: output interfaces. Three output interfaces were compared for next-frame token prediction: (A) the standard arrangement of K independent softmax heads; (B) a *bundle head*, a single linear map onto $\bigoplus_k r_k \otimes a_k$, decoded by K parallel unbind-and-cleanup operations, so that all K tokens of a frame are emitted from one projection without serialization; and (C) a *product head*, a single map onto $a_1 \otimes \cdots \otimes a_K$ carrying the previous frame's product code as streaming state, decoded by the resonator of Algorithm 3 warm-started at the previous frame's factors.

interface	per-stage accuracy	exact frame	decode (ms/frame)
(A) K softmax heads	0.760, 0.606, 0.574	0.491	0.001
(B) bundle head, one projection	0.748, 0.607, 0.577	0.499	0.046
(C) product head + resonator	0.745, 0.596, 0.562	0.477	3.69
copy-previous-frame baseline	0.748, 0.607, 0.577	0.499	—

The three interfaces are within roughly two points of one another on exact frame accuracy, which is the parity result the interface argument needs: a single projection decoded by K parallel unbindings loses nothing relative to K separate heads, and the resonator interface, at a product space of 16^3, remains within 2.2 points while decoding in 3.7 ms per frame of unoptimized array code—inside a 20 ms real-time frame budget with an order of magnitude of headroom.

The result must nonetheless be read with a caveat that the last row makes unavoidable. The bundle head reproduces the copy-previous-frame baseline *exactly*, to three decimals on every stage;

only the multi-head interface exceeds it, and only on the coarsest stage (0.760 against 0.748). At this scale, with a linear predictor on a four-frame window, the prediction problem is close to degenerate: copying the previous frame is very nearly optimal, and the heads largely learn to do that. What the experiment therefore establishes is interface *parity* under matched conditions—no interface destroys information the others retain—and the feasibility of warm-started resonator decoding within a real-time budget. It does not establish that any interface predicts well, and it cannot: that question requires a competent generative model, not a linear probe.

G2: history interfaces. The second claim concerns context. Stage-one token prediction was compared across four context representations: a local window alone; the window plus a fixed-width ($D = 512$) hypervector summary of all earlier frames, with coarse tokens role-bound by temporal bucket; the window plus the full dense history; and the window plus oracle identity labels (digit and speaker), which upper-bounds what any identity-like conditioning can supply.

	horizon $h = 1$		horizon $h = 3$	
context	accuracy	NLL	accuracy	NLL
window only	0.752	0.884	0.567	1.489
window + HDC summary	0.722	0.966	0.570	1.459
window + dense history	0.733	0.901	0.589	1.351
window + oracle identity	0.753	0.867	0.575	1.433

At the one-step horizon there is nothing to compress: the oracle ceiling improves on the window by 0.001 accuracy and 0.017 nats, so every added context channel is noise, and the HDC summary duly hurts. At the three-step horizon a signal appears and the ordering is informative: the fixed-width summary improves on the window (1.459 against 1.489 nats) and recovers roughly half the improvement available from oracle identity, while dense history does better still (1.351), indicating that distant frames carry acoustic information beyond the identity-like content a coarse summary preserves. This is the factorization defect of Definition 1 appearing in a generative setting: what the summary discards is precisely what a coarse token-level abstraction cannot represent.

The decisive limitation is one the experiment cannot escape by tuning. The whole argument for a fixed-width history is asymptotic—constant cost per segment against attention cost growing with stored context—and on half-second utterances there is no long context to compress. At forty frames of history the compressed and dense representations are comparable in size, so the regime in which the exchange pays cannot manifest. Establishing it requires long-form audio and a pretrained generative model, where the appropriate protocol is the three-readout measurement used throughout this section: score a fixed evaluation span under full context, under a truncated window, and under the truncated window plus a fixed-width summary of the remainder, and report what fraction of the long-context value the summary recovers. That measurement is the natural next step, and it is beyond toy scale by construction.

What this licenses. Taken together, G1 and G2 support a narrow claim and refute a broad one. The narrow claim is that the output interface can be replaced by a single role-bound projection with parallel cleanup at no measured accuracy cost, and that temporal persistence supplies basin entry cheaply enough for warm-started resonator decoding to run inside a real-time frame budget: both are properties of the interface, and both were measured under matched conditions. The broad claim—that hypervector memory reduces the cost of audio generation—is not established here and is not the kind of claim a toy can establish. What the toy does is identify where the remaining

risk lives: not in the algebra, which behaves as the theory says, but in the factorization defect of any coarse summary of distant context, which must be measured against a competent model on long-form audio before the compute argument of Section 5.5 can be cashed.

11.10 Takeaways

Every quantitative claim of Section 5 is visible at toy scale: the $D \propto k \log M$ single-step law, the necessity of level-dictionary cleanup for deep descent over incoherent dictionaries, the coherence penalty for near-duplicate dictionaries with its normalization-dependent exponent, the polynomial cold-start operational capacity of the resonator together with its rescue by cue-based basin entry, and the loss and recovery of relation orientation. Just as importantly, the study demonstrates the intended methodological loop of Section 7 in miniature: a quantitative prediction (the k^2 law) was tested, found too coarse for one code family, and refined into a sharper, still-falsifiable claim (the arcsine-law $k^{3/2}$ regime) that the full experimental program of Section 10 can now target. The real-data example adds three lessons that synthetic data cannot: learned effective dictionaries really are small (here, 1.5% of the formal product space) but pervasively coherent (99.6% of entries with near-duplicate neighbors), so the coherence-aware form of the bounds is the practically operative one; cleanup is safe when query-time encoding is consistent with the stored dictionary, but under encoder noise it can become net harmful below the coherence-aware dimension budget at shallow depth and low load, so encoder–dictionary consistency is a precondition the design must either guarantee or budget dimension for; and the three-readout defect decomposition cleanly localizes real-task error between the factorization and the code. The audio study then shows that these conclusions are not modality-specific—the dictionary census, coherence picture, and descent behavior replicate on real speech, with dictionary coverage emerging as the honest new cost—and it contributes the sharpest single number in the program: a role-free commutative code assigns an utterance and its time-reversal *identical* vectors (cosine exactly +1.000), while ordered roles render the reversal nearly as dissimilar as an unrelated utterance, so temporal and causal direction survive in an HDC memory precisely to the extent that the ordering convention is enforced. The generative study finally turns the theory toward synthesis and returns a deliberately mixed verdict: output-interface parity and real-time warm-started resonator decoding are demonstrated under matched conditions, while the history-compression argument is shown to be untestable at toy scale and is left as a defect measurement against a competent long-form model—which is the appropriate outcome, since the capacity law was never a promise that compression is free, only an accounting of what it costs.

12 Application regimes

Video is arguably the strongest first DNN target because object persistence and time-local dynamics naturally induce bounded local structure. Language is also interesting, especially for retrieval and mechanistic features, but it requires careful task restriction. Static images are a useful intermediate case when the encoder provides multiscale VQ tokens, slots, or scene graphs. Columnar causal continual-learning systems are a particularly strong architectural target because their supports, columns, shells, and certificates already mirror HDC roles, codebooks, and local-load budgets.

13 Discussion

What the theory says. The dimension needed for local query fidelity is controlled by local load, effective codebook size, dictionary coherence, cleanup margin, binding distortion, depth,

Domain	Learned factors	Good queries	Main risk
Video	objects, time, pose, velocity, relations, events	tracking, event retrieval, action, anomaly	occlusion and global context
Images	patches, VQ tokens, slots, scene graph nodes	object/property lookup, scene classification, localized retrieval	texture/style defect
Language	tokens, spans, entities, syntax roles, sparse LM features	retrieval, probes, structured QA, routing	long-range pragmatics and generation defect
Causal latents	mechanisms, interventions, parents, state transitions	counterfactual lookup, planning features, invariant prediction	identifiability assumptions
ColBaC/HBCML	columns, shells, microcolumns, certificates, supports	support retrieval, shell hygiene, continual reuse, option memory	premature discretization and dense bridge load
Episodic world models	actors, roles, goals, affect, affordances, evidence anchors, causal and temporal links	partial recall, slot completion, counterfactual judgment, schema consolidation	holistic context defect and unbounded relation load
Relational data	schema roles, records, keys, foreign-key trees	lookup, aggregation, compression	cross-edge load

Table 1: Natural regimes for learned factor graph HDC codes. The method is best viewed as a task-preserving cache of structured latents.

supported query count, and search budget. It is not controlled by raw data size except through those quantities. This is the clean version of the intuition that hierarchical factorization prevents every query from paying for every leaf. The coherence factor deserves emphasis because it is where the idealized random-atom story most often breaks in practice: recursively built dictionaries contain near-duplicates, near-duplicates cost $k^{3/2}$ (sign-quantized codes) to k^2 (linear and phasor codes) rather than k, and the honest options are to merge them (accepting per-path error semantics), separate them (by summary channels or training pressure), or pay the dimension.

What the theory does not say. It does not say that one vector losslessly stores arbitrary images, videos, documents, or agent states. It does not say that resonators converge from any initialization; from cold start, operational resonator capacity grows polynomially in D [7], and the exponential-to-linear search claim is conditional on basin entry, whose costs are charged. It does not say that arbitrary root cosine similarity computes arbitrary tree statistics. It does not say that HDC local lookup beats a well-engineered indexed tree asymptotically. It says that fixed-width HDC can be an efficient task-preserving cache when the upstream representation has low defect, low local load, good margins, low coherence among task-distinguished states, and reusable state dictionaries.

The HDC layer as a learning guide. The most interesting idea in this paper is that the HDC layer is not merely a post-hoc compressor. Its requirements can be fed back into learning. It tells the upstream system which factorizations are useful: those that make each supported query depend on a small number of well-separated role-bound factors, with little task-relevant information left outside the factor graph. This is a different objective from generic disentanglement or reconstruction. It is capacity-aware, query-relative, and operational.

R-HMH as episodic memory. The episodic-memory extension makes the same point in cognitive terms. Episodes are useful to the extent that the agent can recall and recombine their roles, causes, affordances, outcomes, and evidence anchors—with causes and temporal orderings kept directed, which the encoding must arrange explicitly under commutative binding. R-HMH stores an episode as a factorized modular hypervector graph rather than as a flat trace, and resonator cleanup supplies a mechanism for partial completion of missing roles or product factors, with partial cues doubling as basin-entering warm starts. The price is the same as elsewhere: if the supported episodic queries require holistic context not captured by bounded-load factors, the factorization defect dominates.

ColBaC as a natural host. ColBaC already contains sparse support selection, typed columns, reusable shells, internal certificates, and local teachers. HDC adds an algebraic memory and query language for these components. In this setting, HDC can regularize column contents, enrich certificates, warm-start support selection, guide shell promotion, detect bridge overload, and store episodic support memories. The architecture therefore provides a concrete testbed for HDC-guided factorization, not just an application domain.

Streaming generation as a target regime. The compute model of Section 5.5 is stated against named baselines, and autoregressive generation over a factored discrete codec instantiates two of them at once: brute-force search over a product of codebooks at the output, and dense attention over stored context at the input. The generative study of Section 11.9 suggests the two should be treated differently. The output side is an interface question, settled by matched comparison and

favorable at toy scale: one role-bound projection with parallel cleanup matches independent heads, and streaming temporal persistence supplies the basin entry that Corollary 1 requires, cheaply enough to decode in real time. The input side is not an interface question at all but a defect question, and therefore cannot be settled by algebra: whether a fixed-width summary of distant context can replace attention over it depends entirely on how much task-relevant information a coarse abstraction of that context discards. This is the general lesson of Eq. (2) in an applied setting. Where the substitution is exact-by-construction, the theory delivers; where it rests on abstraction, the defect must be measured, and measured against a model competent enough for the measurement to mean something.

Relation to rate-distortion-compute. The code lies on a rate-distortion-compute surface. Smaller D reduces storage and nominal HDC compute, but increases cleanup and factorization error. Once D exceeds the local capacity threshold, the remaining loss is the task-specific factorization defect. The decisive empirical plot is task accuracy versus D, with the saturation floor compared to the full neural or algorithmic state.

Best near-term claim. A conservative and useful thesis is:

> Structured learners expose or learn reusable local factors; resonator-HDC codes provide a compact, compositional, queryable cache of those factors; and the HDC capacity law can guide the learner toward factorizations that are cheaper to store, retrieve, audit, and reuse.

This claim applies to DNNs, ColBaC-style causal columns, symbolic algorithms, program analyzers, database engines, and agentic option memories whenever their internal structure is aligned with the dataset and task structure.

14 Conclusion

We presented a query-limited theory of resonator-factored hierarchical hypervector codes for approximately factorized learned structures. The framework uses explicit or factored level dictionaries, ordered roles, task-specific factorization defect, coherence-aware cleanup bounds, basin-conditional resonator margins, quasi-isometric binding requirements, and honest storage and compute accounting. These restrictions are not incidental. They are the conditions under which an HDC code can be a compact and efficient surrogate for a structured latent graph without violating entropy bounds, hiding search costs, or pretending that structurally correlated dictionaries behave like random ones.

The paper's main conceptual move is to make the HDC layer bidirectional. It is not only a cache placed after a DNN or structured algorithm. It is also a source of pressure on the upstream learner. The HDC capacity law says what kind of factorization is valuable: low-load, role-stable, high-margin, low-confusability, low-coherence, product-recoverable, intervention-local factors that preserve the supported queries and leave little task-relevant information in the residual. These requirements can be turned into losses, audits, and live diagnostics.

The same bidirectional move applies to episodic memory and world modeling. By combining resonator-HDC with HMH, an episode can be represented as a resonantly decodable modular factor graph rather than as a monolithic remembered vector. Actors, objects, actions, goals, places, times, affective states, affordance contexts, causal links, outcomes, evidence anchors, and invoked skills can be stored in named ontology-aligned slots, with directed relations kept oriented by argument-position permutation. Partial cues can then retrieve episodes, resonator cleanup can complete

missing factors, counterfactual factor swaps can train neural judgment models, and repeated retrieval can consolidate experiences into semantic schemas, affordance templates, social scripts, and options.

The ColBaC/HBCML instantiation makes this especially concrete. HBCML supplies a two-level modular learning substrate with sparse supports, internal probabilistic states, certificates, and local teachers. ColBaC supplies columns, typed microcolumns, hard kernels, adaptable shells, and support audits. HDC turns these into addressable role-filler structures: column codes, certificate hypervectors, support memories, option memories, R-HMH episodic traces, and shell-hygiene criteria. The result is a plausible synthesis: ColBaC learns and protects causal modules, while HDC makes their reusable contents compact, compositional, queryable, auditable, and capacity-aware.

The decisive empirical question is therefore not whether natural data is exactly factorized. It is whether a learner can expose a low-defect, bounded-load, low-coherence factor graph for the tasks one wants to answer, and whether HDC-guided learning makes that factor graph easier to store, query, reuse, and protect over time.

References

[1] P. Kanerva. Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors. *Cognitive Computation*, 1:139–159, 2009.

[2] T. A. Plate. Holographic reduced representations. *IEEE Transactions on Neural Networks*, 6(3):623–641, 1995.

[3] P. Smolensky. Tensor product variable binding and the representation of symbolic structures in connectionist systems. *Artificial Intelligence*, 46(1–2):159–216, 1990.

[4] D. Kleyko, M. Davies, E. P. Frady, P. Kanerva, S. J. Kent, B. A. Olshausen, E. Osipov, J. M. Rabaey, D. A. Rachkovskij, A. Rahimi, and F. T. Sommer. Vector symbolic architectures as a computing framework for emerging hardware. *Proceedings of the IEEE*, 2022.

[5] A. Thomas, S. Dasgupta, and T. Rosing. A theoretical perspective on hyperdimensional computing. *Journal of Artificial Intelligence Research*, 72:215–249, 2021.

[6] E. P. Frady, S. J. Kent, B. A. Olshausen, and F. T. Sommer. Resonator networks, 1: An efficient solution for factoring high-dimensional, distributed representations of data structures. *Neural Computation*, 32(12):2311–2331, 2020.

[7] S. J. Kent, E. P. Frady, F. T. Sommer, and B. A. Olshausen. Resonator networks, 2: Factorization performance and capacity compared to optimization-based methods. *Neural Computation*, 32(12):2332–2388, 2020.

[8] Y. Bengio, A. Courville, and P. Vincent. Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8):1798–1828, 2013.

[9] A. van den Oord, O. Vinyals, and K. Kavukcuoglu. Neural discrete representation learning. *Advances in Neural Information Processing Systems*, 2017.

[10] A. Razavi, A. van den Oord, and O. Vinyals. Generating diverse high-fidelity images with VQ-VAE-2. *Advances in Neural Information Processing Systems*, 2019.

[11] F. Locatello, D. Weissenborn, T. Unterthiner, A. Mahendran, G. Heigold, J. Uszkoreit, A. Dosovitskiy, and T. Kipf. Object-centric learning with slot attention. *Advances in Neural Information Processing Systems*, 2020.

[12] F. Locatello, S. Bauer, M. Lucic, G. Raetsch, S. Gelly, B. Schoelkopf, and O. Bachem. Challenging common assumptions in the unsupervised learning of disentangled representations. *International Conference on Machine Learning*, 2019.

[13] J. Brehmer, P. de Haan, P. Lippe, and T. Cohen. Weakly supervised causal representation learning. *Advances in Neural Information Processing Systems*, 2022.

[14] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv:2309.08600*, 2023.

[15] T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLean, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.

[16] J. Snaider and S. Franklin. Modular composite representation. *Cognitive Computation*, 6(3):510–527, 2014.

[17] B. Goertzel. Hierarchical Modular Hypervectors: Baking type-based hierarchical embeddings into modular compositional hypervectors. Manuscript, 2025.

[18] B. Goertzel. PRIMUS world modeling with symbolic, HMH, and dense neural representations. Manuscript, 2026.

[19] B. Goertzel. Causal coding and the general causal-continual-learning theorem. Manuscript, 2025.

[20] B. Goertzel. Toward a general theory of hierarchical Bayesian causal modular learning. Manuscript, 2026.

[21] B. Goertzel. Columnar Bayesian causal coding networks: A two-level architecture for causal modularity, continual learning, universality, and wavelet-transformer integration. Manuscript, 2026.

[22] B. Goertzel. Teacher-first columnar control for continual learning: A two-level architecture with exact support search. Manuscript, 2026.

[23] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.

[24] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *ICLR*, 2017.

[25] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[26] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell. Progressive neural networks. *arXiv:1606.04671*, 2016.